Alma Mater Studiorum · University of Bologna

Department of Computer Science and Engineering
Master's Degree in Computer Science

ZERONETHERMIND: ENGINEERING A STATELESS EXECUTION LAYER FOR ETHEREUM USING ZERO-KNOWLEDGE PROOFS

Supervisor:
Prof. Marco Di Felice

Submitted by:
Manuel Arto

Co-supervisor:
Dr. Luca Sciullo

March 2026 Session
Academic Year 2024/2025

Abstract

Ethereum's overall speed is constrained by the computational capabilities of its slowest node. To preserve a secure and decentralized environment, the protocol currently requires each validator to re-execute every transaction and maintain a state database on the order of terabytes. This approach creates a bottleneck with $O(N)$ complexity relative to transaction volume and triggers the architectural criticality known as State Bloat, progressively pushing consumer hardware away from active participation in the network. This thesis presents ZeroNethermind, a proof-of-concept that transforms the Execution Layer client Nethermind into a stateless validator, introducing a cross-language FFI bridge between C#/.NET and Rust that bypasses native EVM execution through the verification of cryptographic proofs generated by five independent zkVMs. The experimental campaigns, conducted in real time on the Ethereum Mainnet and in a controlled devnet environment, confirm a verification time that is nearly constant and independent of block complexity, with significant reductions in resource usage. By delegating the execution workload to specialized provers and enabling verification on consumer devices, ZeroNethermind demonstrates the feasibility of a new validation model, contributing to the engineering foundations for scaling Layer 1 without compromising network decentralization.

Contents

Chapter 1

Introduction

The fundamental promise of decentralized blockchain networks lies in the democratization of trust and systemic resistance to censorship. This algorithmic *freedom*, however, clashes with rigorous physical and architectural limitations. In the Ethereum ecosystem, maintaining network integrity in a trustless environment currently requires every validator node to natively re-execute every single transaction. Consequently, the overall network throughput is constrained by the computational capabilities of the slowest node within the system.

This architectural bottleneck imposes a re-execution cost of $O(N_{gas})$, proportional to the aggregate complexity of a block's transactions. Increasing the Gas Limit to process a higher volume of transactions would inevitably exclude consumer-grade hardware. Compounding this issue is the architectural criticality known as State Bloat, the uncontrolled expansion of the global state database.

It is essential to clarify that the primary bottleneck in processing and validating blocks on Ethereum is not computational in nature (CPU-bound), but rather related to data access (I/O-bound). The execution of a single transaction within the EVM requires continuous and fragmented read and write operations on the state database, which must accurately track the totality of account balances and smart contract

memory slots.

Currently, this data layer requires nodes to locally maintain archives on the order of terabytes. As the state continues to grow, hardware requirements intensify: the limit is no longer dictated by storage capacity, but by the need for memories with extremely high performance in terms of IOPS (Input/Output Operations Per Second) to sustain the pace of execution. This drift would push validation toward a network dominated by centralized data centers, threatening the very foundation of decentralization and censorship resistance.

With the transition from the "The Merge" architectural phase to the future "The Verge" roadmap[2], the Ethereum ecosystem is actively aiming to mitigate this issue. Its long-term objective is to achieve *statelessness*, lowering hardware requirements to the point of enabling full chain validation on ultra-low-power devices such as smartphones or smartwatches.

Historically, the evolution of cryptography has charted the roadmap for the security of global digital infrastructures: the introduction of hash functions, asymmetric key cryptography, and digital signatures laid the foundations for authentication and value transfer without intermediaries. The adoption of Zero-Knowledge (ZK) cryptography represents the next evolutionary step.

Traditional blockchain architectures are inherently "symmetric": every node is forced to duplicate the exact same computational effort as the node that produced the block. The application of Zero-Knowledge Proofs (ZKPs) imposes a radical paradigm shift: the effort to generate the block and the corresponding cryptographic proof remains onerous, but the subsequent verification becomes computationally negligible. In a ZKP-based ecosystem, the computational validation of a block containing ten thousand transactions requires the same amount of time needed to verify a block containing a single transaction.

This principle is materialized in the "Beast vs. Fort" topological

model[6]. On one side, centralized "Beast Mode" infrastructures (Provers/Builders) equipped with ultra-high-performance hardware clusters generate the blocks and their corresponding cryptographic proofs. On the other side, validators in "Fort Mode" operate on consumer-grade hardware, removing the burden of execution and limiting themselves to certifying the mathematical validity of the proof and the block headers, guaranteeing cryptographic security in defense of the network.

The primary objective of this thesis is the engineering of ZeroNethermind: a Proof-of-Concept designed to transform the Execution Layer client Nethermind into a "Fort Mode" node, aimed at validating Mainnet (L1) blocks using ZKPs, completely freeing itself from the need to maintain a local state database.

This work makes the following contributions to the Ethereum client landscape:

- **Implementation of the "Double Zero" Paradigm:** Practical demonstration of the feasibility of a *stateless validator* (zero-state) that integrates zero-knowledge to replace re-execution within a production-ready client.
- **Cross-Language Interoperability:** Development of a high-performance FFI (Foreign Function Interface) bridge between the C# (.NET) environment and Rust cryptographic libraries.
- **Live L1 Validation:** Interception of the Engine API Data Flow to validate in real time the blocks at the tip of the Mainnet by delegating proof generation to external infrastructures.
- **Quantitative and Performance Analysis:** Empirical evaluation of cryptographic verification times compared to native EVM execution and analysis of the reduction in resource footprint in a private Ethereum devnet.

The scope of the research focuses on the Execution Layer (EL), the dynamics of Engine API interception, and the architectural feasibility

of the stateless approach. The engineering of the proving infrastructure (Beast Mode) and the local generation of cryptographic proofs are outside the scope of this work. Furthermore, the extended theoretical formulation of the arithmetic circuits underlying ZK protocols is covered only in terms of their practical application for verification.

The document is structured as follows. `Chapter 2` presents the State of the Art, analyzing Ethereum's architecture, the scalability dichotomy between Layer 1 and Layer 2, the stateless client paradigm, and the fundamentals of Zero-Knowledge cryptography, concluding with a review of related work and protocol upgrade proposals. `Chapter 3` describes the design of ZeroNethermind: the "Double Zero" paradigm, the three-layer architecture, and the modified data flow compared to standard validation. `Chapter 4` documents the implementation conducted through three distinct incremental phases. `Chapter 5` presents the experimental validation on two fronts: a live Demo on the Ethereum Mainnet and a controlled environment based on the Kurtosis devnet for comparative analysis. `Chapter 6` closes the document with conclusions, PoC limitations, and future development perspectives within the context of Ethereum's roadmap.

Chapter 2

State of the Art

This chapter aims to outline the technological and scientific context in which this thesis work is situated, providing the necessary foundations to understand the current challenges and emerging solutions in the blockchain ecosystem. The chapter is structured into four main sections: the first analyzes Ethereum's Architecture, defining its fundamental principles, intrinsic performance limitations, and node operation; the second explores the scalability dichotomy (L1 vs L2); the third analyzes the introduction of Stateless Clients within the Ethereum protocol; the fourth section is dedicated to Zero-Knowledge cryptography, the engine of the paradigm shift from computation to verification; finally, the last section reviews related work and the current research landscape.

2.1 Ethereum Architecture

Ethereum is the most globally adopted *general-purpose* blockchain platform for the execution of *smart contracts*. More than merely a network for value transfer, Ethereum was conceived as an infrastructure on which anyone can build decentralized applications (dApps) without requiring permissions (*permissionless*). The three fundamental pillars on which this architecture is based are:

- **Transparency:** All code and transaction history are public and cryptographically verifiable independently.
- **Censorship Resistance:** No central entity, government, or corporation has the authority or technical capability to block a valid transaction or prevent access to the network.
- **User Control:** Through the use of asymmetric key cryptography, users maintain full and sole sovereignty over their funds and data (*self-custody*).

The architectural innovation introduced by Ethereum was the creation of the first "decentralized global computer." Unlike traditional centralized computing paradigms (e.g., cloud services managed by corporate entities), Ethereum's infrastructure is not located in a single physical place. It is a distributed virtual machine that cannot be shut down, reset, or destroyed. Access requires only an Internet connection, and the cryptographic infrastructure guarantees an enormous address space: private keys are derived from a probability space of 2^{256}, from which 160-bit addresses are extracted, virtually guaranteeing inexhaustible space for the entire global population without any practical risk of collisions.

2.1.1 Blockchain Trilemma

Although Ethereum functions as a global computer, it is inherently slower and more expensive than centralized computing. This is not an accidental engineering inefficiency, but a precise architectural choice linked to the "Blockchain Trilemma." This postulate states that a decentralized network must balance three properties — Scalability, Security, and Decentralization — historically being able to maximize only two simultaneously.

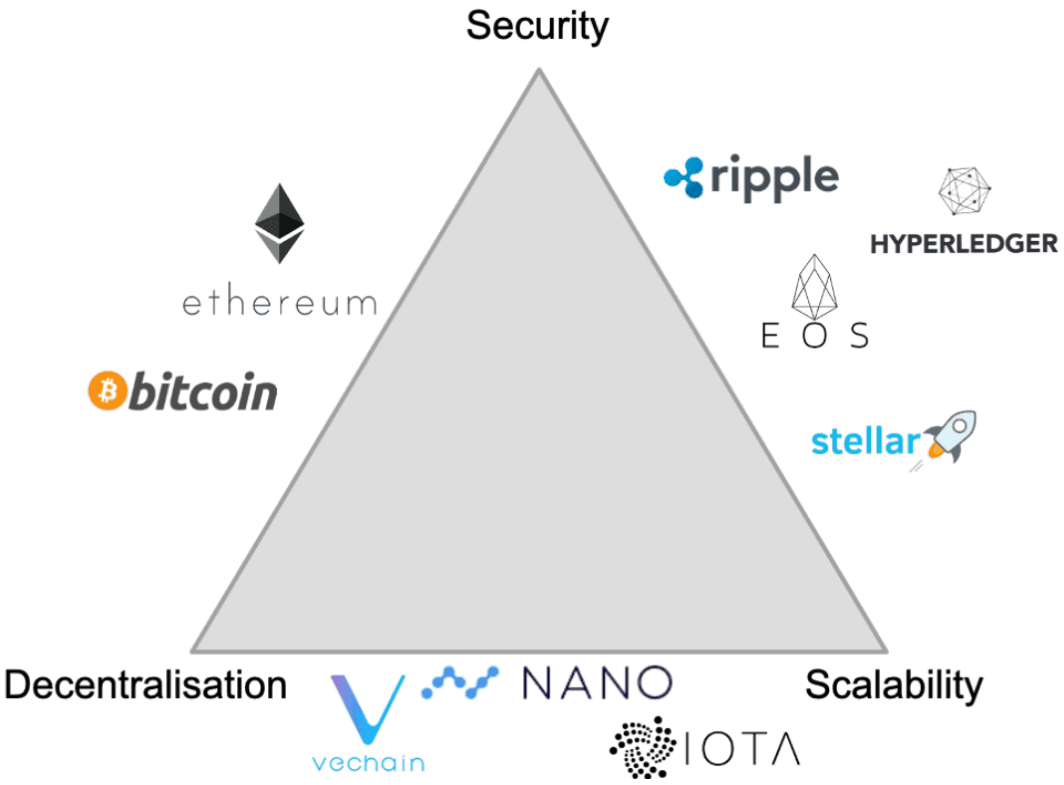

Figure 2.1: Blockchain Trilemma [21]

Ethereum has systematically prioritized Security and Decentralization. The primary bottlenecks are attributable to two factors:

1. **Re-execution:** To ensure that the system is *trustless*, every single validator node in the network must independently re-execute every single transaction to verify its validity. This redundant work imposes an asymptotic cost of $O(N_{gas})$ on execution, making the entire network only as fast as its "slowest node."
2. **Network Latency and Consensus:** Ethereum is a globally distributed P2P network. To allow thousands of nodes to receive data, validate blocks, and reach probabilistic or deterministic agreement on the network state, the protocol imposes a division of time into fixed 12-second *Slots*.

2.1.2 State in Ethereum and the Transition Function

Ethereum fundamentally operates as a complex transaction-based deterministic finite state machine. The concept of "State" is vital: it is the global database containing the up-to-date picture of the network.

The literature distinguishes between *World State* and *Account State*. The *World State* maps each 160-bit address to its respective *Account State*. An *Account State* is composed of four elements: the *nonce* (transaction counter), the *balance* (ETH balance), the *codeHash* (the hash of the contract bytecode, if present), and the *storageRoot* (the root of the tree mapping the contract's internal state variables).

The transition from one valid state to the next is governed by the *State Transition Function* (STF) of the Ethereum Virtual Machine (EVM). As formally defined in the Yellow Paper[27], the application of a transaction T to the pre-existing state σ_t generates a new valid state σ_{t+1}:

$$\sigma_{t+1} \equiv \Upsilon(\sigma_t, T) \tag{2.1}$$

To guarantee the cryptographic integrity of this immense database (which today weighs several hundred gigabytes, exceeding one terabyte for *Archive* nodes), Ethereum does not use a standard relational database, but rather a cryptographic tree called the Merkle Patricia Trie (MPT). The MPT functions as an authenticated database composed of four node types: *Branch* (16-way branching nodes), *Extension* (nodes compressing common paths), *Leaf* (leaf nodes containing the final value), and *Empty* (null nodes). The raw serialized data of this tree is physically persisted to disk through high-performance key-value database engines such as LevelDB or RocksDB. However, the computational and I/O (Input/Output) cost of this architecture is extremely burdensome: for each persistence and update operation, the EVM must cryptographically recalculate hashes starting from the modified leaf node and traversing all the way up to the root. As the MPT grows in size and depth, this authentication mechanism gener-

ates a severe amplification of read and write operations on disk (*read-/write amplification*), reaching up to 64 database accesses for a single operation in the worst case. This makes the $O(N_{gas})$ re-execution of transactions a predominantly *I/O-bound* problem (limited by latency and sequential/random storage access) rather than purely *CPU-bound*. The hash of the new global tree root (*State Root*) is finally calculated and inserted into each new block header to seal its validity.

2.1.3 Gas Limit and Bottlenecks

The *Gas* is the unit of measurement for the computational effort required to perform an operation in the EVM. Since Ethereum imposes re-execution on all nodes, the network must protect itself from the Halting Problem and unlimited resource consumption. This is achieved through the *Block Gas Limit*, an absolute ceiling on the amount of computation that can be performed in a single block (currently around 36 million gas, slowly evolving and the subject of intense debates for increases up to 60 million).

"Simply" increasing the Gas Limit would entail a drastic increase in computational workload, bandwidth, and RAM and storage requirements (NVMe SSD). The worsening of *State Bloat* — the permanent accumulation of state data — and the increased validation burden would inevitably push consumer-grade hardware out of the network, delegating consensus to professional data centers and destroying the credible neutrality of the infrastructure.

2.1.4 Execution Layer (EL) vs Consensus Layer (CL)

Following "The Merge" upgrade (transition to the Proof-of-Stake mechanism), the architecture of Ethereum clients underwent a profound refactoring, separating responsibilities into two software modules executed in tandem:

- **Execution Layer (EL):** This layer (e.g., Nethermind, Geth) is

the computational "engine." The EL maintains the *World State* MPT, manages the local *mempool* of unconfirmed transactions, and natively executes the EVM to apply the state transition function.

- **Consensus Layer (CL):** Also known as the *Beacon Node* (e.g., Lighthouse, Prysm), this layer is agnostic to *smart contract* execution. Its task is to govern the Proof-of-Stake consensus protocol: it coordinates validator turns, propagates blocks via gossip network, aggregates signatures (*attestations*), and applies the Fork Choice Rule to establish the canonical chain.

These two layers communicate bidirectionally, securely, and in a standardized manner through a suite of JSON-RPC interfaces known as the Engine API.

2.1.5 Nethermind Client

Nethermind is one of the main Execution Layer clients for Ethereum, developed entirely in C#/.NET. The project entered production in 2019 and represents, alongside Geth (Go Ethereum) and Reth (Rust Ethereum), one of the three most widely deployed EL clients on the mainnet, actively contributing to protocol diversity.

The hardware requirements[20] to run a Nethermind node in full validation are significant and continuously growing:

- **CPU:** 4 or more cores.
- **RAM:** At least 16 GB (32 GB recommended for optimal performance).
- **Storage:** Over 2 TB of high-performance NVMe SSD (min 10,000 IOPS).

Nethermind uses RocksDB[19], a high-performance key-value database engine, for disk persistence. The data is organized into approximately

six categories (State, Receipts, Blocks, Headers, Code, Metadata), with the state database alone exceeding 150 GB. The entire database, including block history and receipts, reaches and surpasses 1 TB. This massive storage footprint, combined with the need for high IOPS, represents the primary barrier that risks excluding consumer-grade hardware from active participation in the network in the future.

2.1.6 Block Validation Flow and the Engine API

The process of validating a new block represents the moment when the computational burden physically falls on the node. The communication dynamics between EL and CL are the fulcrum upon which ZeroNethermind operates. The lifecycle follows these architectural steps:

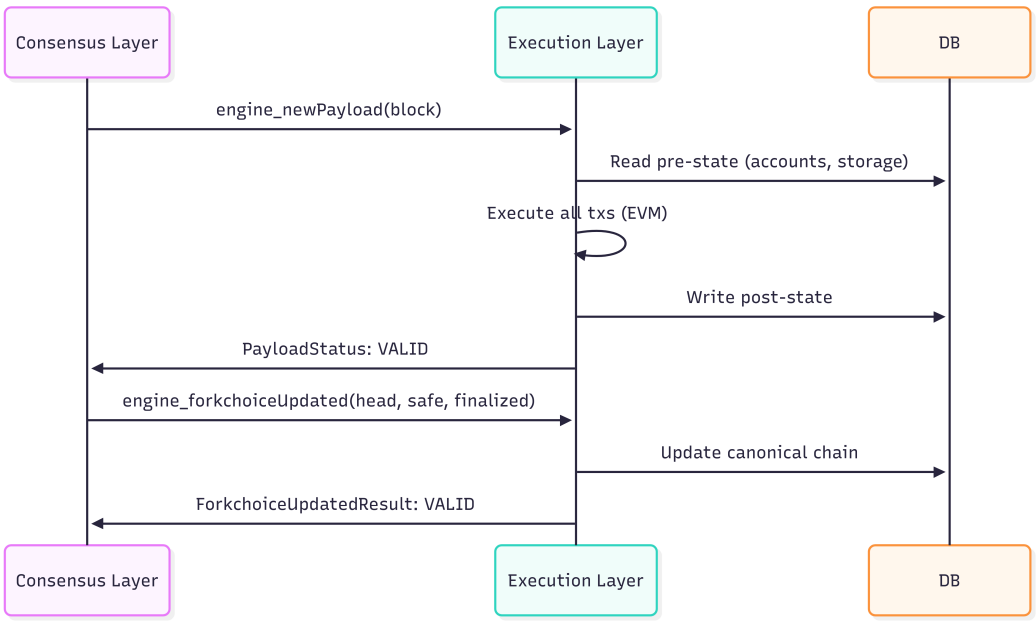

Figure 2.2: Standard Validation Flow

1. **Mempool and Construction:** Transactions sent by users populate the *mempool* of the Execution Layer. The node selected as

the *Proposer* for the current slot aggregates the transactions into an *ExecutionPayload* and encapsulates it in a *Beacon Block*.

2. **Propagation (CL):** The Beacon block is broadcast through the Consensus Layer's P2P network.
3. **Reception and Query (Engine API):** When a remote node receives the block, its CL extracts the execution payload and sends it to its local EL by invoking the RPC method `engine_newPayload`.
4. **Native Re-execution (EL):** The EL must rigorously verify the block. It loads the antecedent state from its own disk, checks the validity of the base fee, gas availability, nonces, and initial balances, and sequentially executes each transaction within the EVM. Finally, it recalculates the MPT root.
5. **Validation Outcome:** If the recalculated *State Root* matches the one declared by the Proposer, the EL responds to the CL with the status `VALID`.
6. **Fork Choice Update:** Upon confirmation of execution validity, the CL sends the `engine_forkchoiceUpdated` command, instructing the EL to designate this block as the new *head* of the canonical chain.

2.2 Scalability: L2 vs. L1

Historically, in response to the severe intrinsic transaction-per-second limit dictated by the N-of-N re-execution model, the research community has delegated the computational workload to secondary networks. This approach gave rise to the so-called "Rollup-Centric" roadmap. In this paradigm, Layer 1 (L1) sacrifices execution scalability to maintain an exclusive role as the layer of maximum security and data availability guarantee. Massive transaction processing is instead moved *off-chain* to Layer 2 (L2), known as *Rollups*.

There are two main approaches to ensure that execution performed on L2 is valid when finalized on L1:

- **Optimistic Rollups:** Assume by default that all processed transactions are correct. Security is guaranteed by a dispute resolution mechanism (*Fraud Proofs*), which however introduces significant latency (typically 7 days) before transactions reach irrefutable finality on L1.
- **ZK-Rollups:** Leverage *Validity Proofs* (cryptographic proofs such as SNARKs or STARKs) generated *off-chain* to certify the mathematical integrity of each batch of transactions. This approach guarantees correctness a priori and allows settlement on L1 constrained solely by the time to generate the cryptographic proof.

2.2.1 Criticalities of Current L2s

Despite the proliferation of L2 solutions, their current architecture presents systemic criticalities that contradict the decentralized ethos of L1. The assumption of passively inheriting Ethereum's security often conceals the use of *Training Wheels*: centralized safeguard mechanisms that grant a few actors the power to alter the protocol.

Analyzing the *L2Beat* classification framework[18], it can be observed that the vast majority of current L2s are at "Stage 0" or "Stage 1" of decentralized maturity. These networks rely on *Sequencers* operated by single corporate entities and *Security Councils* governed by *multisig* keys capable of overriding execution or forcing upgrades to the validation smart contracts.

This situation has led to a profound theoretical recalibration: an L2 that does not offer solid *trustless* cryptographic guarantees is not truly scaling Ethereum, but is operating as an independent L1 disguised as an L2 through a centralized bridge. To formalize this concept, the *Ethereum Settlement Score* (ESS) was introduced, a metric that

quantifies the degree to which an L2 irrefutably uses the base layer to finalize transactions.

At the ecosystem level, this horizontal proliferation of imperfect Rollups has furthermore drastically fragmented liquidity and destroyed *Synchronous Composability* — the ability of smart contracts to interact atomically with each other within the same block, a fundamental property that had fueled the success of Decentralized Finance (DeFi) on Layer 1.

2.2.2 The Paradigm Shift: Scaling L1 through ZK

Overcoming the vulnerabilities of L2s requires a return to empowering Layer 1, abandoning the paradigm in which "everyone computes everything" in favor of a model in which "one computes, all verify." This represents the architectural leap from redundant computation (N-of-N) to cryptographic verification (1-of-N).

This principle was formalized by Ethereum Foundation researcher Justin Drake[6] through the architectural dichotomy known as Beast Mode vs. Fort Mode:

- **Beast Mode:** Involves outsourcing massive execution and cryptographic proof generation to *Provers* and *Builders* operating on data-center-grade infrastructure. The performance objective is to push the L1 processing limit toward the *GigaGas/s* frontier (up to 10,000 TPS), surpassing the limits of general-purpose servers.
- **Fort Mode:** Represents the bastion of decentralization. Validators cease natively executing the EVM to become "ultra-light" validators. Thanks to the computational asymmetry provided by ZK cryptography, these nodes can operate on consumer hardware (laptops, smartphones, or even smartwatches) by limiting themselves to mathematically certifying the proofs generated by the "Beasts."

Characteristic	**Beast Mode**	**Fort Mode** (ZeroNethermind / Validators)
Role	Performs heavy computations and generates cryptographic proofs.	Secures the chain by verifying proofs
Hardware	Specialized Hardware.	Consumer laptops, tablets, smartphones
Storage (L1 State)	Terabytes	0 GB (MemDB)
Computational Complexity	$O(N_{gas})$ - proportional to transaction volume	$O(1)$
Network Objective	Scale Ethereum to 10k+ TPS (GigaGas).	Uncompromising censorship resistance and security

Table 2.1: Architectural comparison between the Beast Mode and Fort Mode paradigms

The transition toward this architecture is planned in two phases: a first phase of *optional proofs*[10], in which nodes can voluntarily choose to verify blocks via ZK proofs rather than re-executing them, followed by a phase of *mandatory proofs*, in which cryptographic verification becomes the default mechanism and re-execution an option for specialized roles (RPC, block building). ZeroNethermind anticipates the first phase, demonstrating that an EL client can already operate in proof-only mode today.

2.3 Stateless Clients

One of the most powerful properties of a blockchain network is the ability for anyone to run a node on their own computer and independently verify the correctness of the chain. Even if the vast majority of consensus nodes were to agree on malicious rules, a node in full validation would categorically reject the compromised chain. However, for this security dynamic to remain valid, running a node must remain accessible to a critical mass of users.

Today, running a node on consumer hardware is possible, but increas-

ingly complex due to the volume of data that must be stored. The Ethereum roadmap phase called "The Verge" aims to revolutionize this aspect, making validation so lightweight that it could be run by default on mobile devices, browser wallets, or even smartwatches.

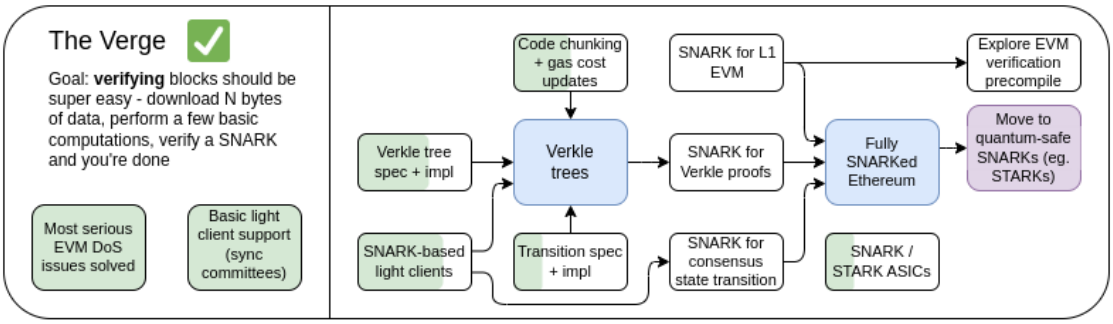

Figure 2.3: The Verge, 2023 roadmap [2]

The fundamental problem that The Verge seeks to solve is the uncontrolled growth of state data. Currently, an Ethereum client must store hundreds of gigabytes of data to validate blocks, with an increase of approximately 30 GB of raw data per year, in addition to the metadata needed to efficiently update the database. This technological friction extends synchronization times to entire days and disincentivizes the participation of independent nodes.

Originally, The Verge aimed to replace Merkle Patricia Tries with Verkle Trees, data structures that allow much more compact proofs. Today, the vision has expanded to include verification of the entire Ethereum execution through SNARK cryptographic proofs, aiming at two key objectives:

- **Stateless Clients:** Full validation clients and staking nodes will require no more than a handful of gigabytes of storage.
- **Verification on ultra-light devices:** In the long term, chain validation will consist solely of downloading a minimal portion of data and performing the mathematical verification of a SNARK.

2.3.1 Stateless Ethereum

"Stateless Ethereum" represents a radical protocol upgrade in which blocks become self-contained execution units. The node is no longer required to download the entire network state, since all information needed for execution and validation is packaged directly within the block itself through cryptographic proofs (the so-called *witnesses*).

The main benefits of this architecture branch across multiple fronts:

- **Scalability:** By removing the I/O bottleneck related to on-disk database access, validators can process many more transactions, allowing a safe increase in the Gas Limit and therefore a higher TPS (Transactions Per Second).
- **Decentralization:** The reduction of hardware requirements allows new actors to enter network security, paving the way for ultra-light *trustless* clients and flexible dynamics such as *rainbow staking* or the creation of private validation pools.
- **Instant Synchronization:** Not requiring the historical database, a node only needs the current block to actively join the network.

2.3.2 Relationship Between Block Verification and State

To fully understand the relationship between Ethereum's state and block verification, consider a scenario in which a block containing a single transaction must be validated: the transfer of 10 ETH from account A to account B. Omitting secondary details, the basic logic requires verifying that account A has a sufficient balance to cover both the transferred amount and the network fees (*gas fees*). Otherwise, the transaction must fail.

It is crucial to note that, upon receiving a block, an Execution Layer client cannot predict in advance which accounts will be involved. Consequently, in a traditional architecture, the client is forced to maintain in memory the information relating to all existing Ethereum accounts.

The absence of such data would make block verification impossible for an unequipped client, particularly penalizing *stateless* architectures. More complex transactions, such as those invoking *smart contracts*, follow the same logic: clients must have the contract bytecode and the corresponding storage state available, since blocks can theoretically contain transactions that interact with any contract deployed on the network.

The key insight underlying the *stateless* paradigm is that the portion of state actually needed to verify a single block is orders of magnitude smaller than the entire *World State*. There are portions of Ethereum's state that have not been queried in years, yet must still be stored in case a new transaction might require access. This issue tangentially raises the need to implement *State Expiry* mechanisms at the protocol level.

2.3.3 Stateless Validators

For a node to traditionally validate a block, it must possess the antecedent state (pre-state). In a fully-SNARKified model, the Execution Layer Client does not need to execute any EVM code: it simply needs to validate a cryptographic proof attesting to the correct transition from the `pre_state_root` to the `post_state_root` by applying the hash of the current block.

This dynamic enables the creation of **Stateless Validators**. Since they do not possess the complete database, these nodes cannot operate as *Block Builders*, a task that requires full access to the state to simulate transactions and extract value. However, thanks to already existing mechanisms such as *Proposer-Builder Separation* (PBS) or MEV-boost, a stateless client can delegate construction to third parties, receive the candidate block with its corresponding proof, verify its integrity in constant time, and propose it to the network.

2.3.4 Stateless Client Architecture

The engineering advantage of a stateless client lies in its extreme architectural simplicity compared to a full node. By switching to the pure verification paradigm, entire client modules become obsolete or superfluous.

Observing the structure of a stateless node, we notice a drastic reduction in active components:

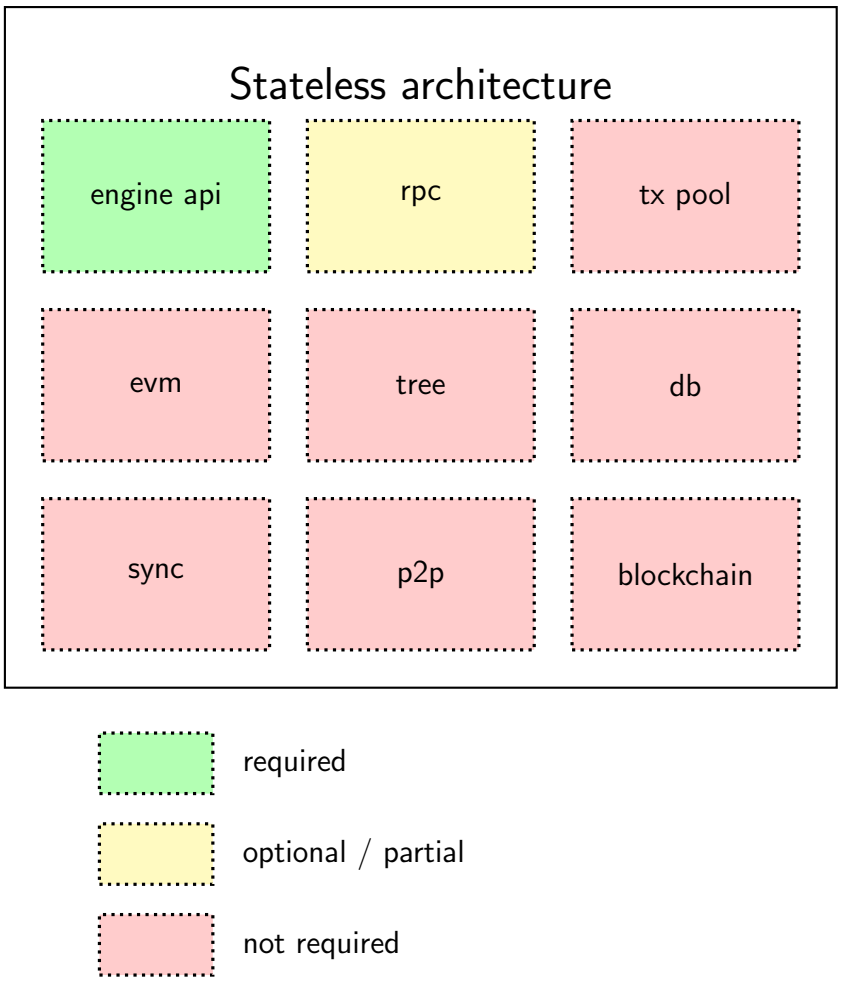

Figure 2.4: Stateless Client Architecture

- **Eliminated Components (Not Required):** The client is streamlined by removing the EVM (as there is no native execution),

the transaction Mempool (*Tx Pool*), the state tree update logic (*Tree*), the disk persistence engine (*DB*), historical synchronization (*Sync*), and the Gossip propagation infrastructure (*P2P* and *Blockchain historical data*).

- **Essential Components (Required):** The *Engine API* communication interface remains the beating heart for interacting with the Consensus Layer.
- **Optional Components:** The *RPC* service can be maintained for limited queries, although it cannot answer queries dependent on the global state.

For the integration of these clients, two main patterns are emerging:

1. **Separate Clients:** The stateless client (CL+EL) connects to *stateful* peers via a network sub-protocol. Execution and verification are decoupled but remain interactive.
2. **Embedded Clients:** A single client combines the consensus logic (CL) with a lightweight embedded EL Verifier (e.g., a zkVM or a stateless executor). This client receives blocks and proofs via the gossip network and verifies them entirely locally, a model that mirrors the architecture implemented in this thesis.

2.4 Zero Knowledge

The evolution of computer cryptography can be divided into two distinct generations. The "First Generation" was characterized by *special-purpose* algorithms, handcrafted to fulfill highly specific tasks, such as hash functions for integrity or digital signatures for transaction authentication. The advent of the "Second Generation" marks the transition to *general-purpose* cryptography. Formally introduced in the late 1980s by cryptographers Goldwasser, Micali, and Rackoff, Zero-Knowledge[26] (ZK) cryptography represents a particularly relevant expression, conferring the unprecedented ability to formulate

cryptographic assertions about entirely arbitrary and Turing-complete computations. A zero-knowledge proof (Zero-Knowledge Proof, ZKP) allows an entity to mathematically prove the truthfulness of a statement without revealing any underlying information or secret data. This technology holds the power to transfer the trust model from centralized institutions or specialized hardware directly to the infallibility of pure mathematics.

At the application level, ZK protocols enable a vast range of cross-cutting use cases. In the domain of digital identity (ZK-ID and ZK-KYC), they allow a user to prove the passing of identity checks (for example, being of legal age) or possession of sufficient funds for a transaction, without exposing their date of birth or actual bank balance. In the institutional financial sector, ZKPs guarantee compliance with anti-money laundering (AML) regulations and sanctions, while simultaneously keeping client positions and trading strategies private. In the distributed cloud computing landscape, they also allow computationally expensive workloads to be delegated to untrusted servers, which return the result accompanied by a mathematical certificate of correct execution verifiable in real time.

It is precisely on this last aspect that Ethereum's transition toward *statelessness* and massive scalability is based, through the adoption of succinct and non-interactive cryptographic proofs (SNARG/SNARK). The founding mathematical principle of these protocols from a scalability perspective lies in the introduction of a sharp computational asymmetry between participating entities: on one side stands the *Prover*, a machine with very high computational capacity tasked with executing the computation and generating the cryptographic proof; on the other side operates the *Verifier*, a node with limited hardware resources whose sole burden is to rigorously check the validity of the received proof.

In the blockchain domain, and particularly in the Ethereum ecosys-

tem, a formal academic clarification is however warranted: the use of the term "Zero-Knowledge" (ZK) often represents a *misnomer* (a terminological inaccuracy). The fundamental property that the so-called "zkEVMs" exploit to break the execution bottleneck is not *privacy* (i.e., the concealment of transaction data), but rather succinctness (*succinctness*). Consequently, at the engineering level, architectures like ZeroNethermind should more correctly be defined as *Succinct EVMs* or *SNARG-EVMs*.

2.4.1 The Interface of a zkVM

From a system integration perspective, a zkVM operates as a cryptographic construct that exposes a standardized interface defined by three main functions:

- `Prepare(f)` $\rightarrow$ `vk`: During the setup phase, the algorithm receives as input the formal description of program `f` (such as the EVM state transition function) and generates a public verification key (*verification key*, `vk`) of extremely reduced size.
- `Prove(f, x)` $\rightarrow$ `(y, proof)`: Function executed by the *Prover*. It computes the output `y` from the input state `x` and simultaneously produces the cryptographic proof (`proof`) attesting to the correct execution of `f(x)`.
- `Verify(vk, x, y, proof)` $\rightarrow \{$`0, 1`$\}$: Function executed by the *Verifier*. Taking charge of the verification key, the public inputs/outputs, and the proof, the algorithm returns a boolean value that certifies or rejects the validity of the transition in sublinear time.

2.4.2 The Three Fundamental Properties

For the network to safely delegate *trustless* execution, the underlying proving system must guarantee three non-negotiable mathematical

properties:

1. **Completeness:** If the computational statement is true and the *Prover* behaves honestly, the *Verifier* will always accept the proof. This property is crucial to preserve the blockchain's *liveness*, avoiding false negatives that would halt chain progress.
2. **Soundness:** If the statement is false, no computationally bounded adversary can generate a valid proof, except with negligible probability. In zkVMs, *Knowledge Soundness* is employed, which requires the *Prover* to not only compute the result but to actually know the entire execution trace (*witness*) in order to construct the proof.
3. **Succinctness:** Represents the keystone for solving the scalability trilemma. The proof size and, most importantly, the computational verification time are sublinear relative to the time needed for native re-execution of the program, settling at constant ($O(1)$) or polylogarithmic resolution times.

2.4.3 Architectural Problems Solved by zkEVMs

The scalability of any blockchain architecture universally clashes with three fundamental constraints: *Compute*, *I/O*, and *Bandwidth*. The introduction of zkEVMs, designed specifically to prove the correct execution of Ethereum's state transition function, intervenes to jointly mitigate these architectural limitations. First, the compute constraint is resolved directly by the very nature of ZK proofs, which compress the computational burden into a nearly instantaneous cryptographic verification. Second, the I/O constraint is resolved indirectly: by not having to re-execute transactions in the EVM, nodes are relieved of the obligation to constantly load and update the enormous local state database, effectively enabling partially or fully *stateless* operations. Finally, the bandwidth problem is addressed by coupling the ZK paradigm with Data Availability Sampling (DAS) mechanisms,

allowing the network to attest to the availability of compressed data and blocks without forcing nodes to integrally download all historical transactions.

In the current protocol, throughput is strictly constrained by the *Block Gas Limit*. An increase would require validators to have increasingly expensive hardware to sustain full re-execution, centralizing consensus. In zkEVMs, instead, a single *builder* executes the entire block and a *prover* generates its cryptographic proof. Validators limit themselves to *proof checking*, an asymptotically cheaper operation. This 1-of-N paradigm allows the gas limit to be raised in total safety: expanding block capacity spreads the high demand across greater availability of space, stabilizing and reducing fee prices.

In parallel, delegating the computational workload to *provers* prevents *finality* delays. Since cryptographic verification requires a constant time independent of block complexity, validators no longer risk missing slots due to excessively slow re-executions. The hardware barrier is thus radically dismantled, even allowing modest devices (such as a Raspberry Pi) to participate in securing the network and preserving its absolute diversity.

By sharply separating the massive execution phase from the validation phase, zkEVMs radically transform Ethereum's scalability model toward a unified architecture. This dichotomy generates a highly competitive execution market that incentivizes the optimization of specialized hardware for *provers*, while simultaneously guaranteeing massive validator diversity through the empowerment of lightweight nodes. The end result is elastic and secure throughput, allowing Layer 1 capacity to grow without ever compromising the decentralization and accessibility of the network.

2.4.4 Ethproofs.org

Ethproofs.org[12] is a *Block Proof Explorer* for Ethereum: a public platform that aggregates ZK cryptographic proofs generated by multiple independent proving teams for Mainnet blocks. Launched by the Ethereum Foundation, ethproofs.org plays a role analogous to that of a traditional block explorer, but specialized in tracking ZK proofs.

The platform, in addition to presenting an interactive website, exposes a public REST API that allows querying the metadata of available proofs for each block and downloading both the binary proofs and the corresponding verification keys (*Verification Keys*, VK).

The Prover Ecosystem

One of the most significant characteristics of ethproofs.org is its nature as a multi-prover aggregator. The platform collects proofs generated by different proving clusters, each based on a distinct zkVM (*Zero-Knowledge Virtual Machine*). Table 2.2 summarizes the main provers currently operational on the platform.

Prover	**zkVM Family**	**Proving System**
Zisk	Polygon Hermez	pil2-proofman
OpenVM	Axiom	STARK (verify-stark)
Pico	Pico Prism	KoalaBear Poseidon2
SP1 Hypercube	Succinct Labs	Compressed STARK
Airbender	Matter Labs (zkSync)	Mersenne31 + Unified Recursion

Table 2.2: Main provers operational on ethproofs.org.

This client diversity is intentional. Multiple teams compete to generate valid proofs in the shortest possible time, foreshadowing the emergence of a true *Prover Market*[16].

2.5 Related Work

The architecture and design principles underlying the project presented have roots in a broad and rapidly evolving line of research, spanning from distributed systems theory to the most recent Ethereum protocol upgrade proposals (EIPs). This section analyzes the foundational literature and projects that serve as a prelude to implementing an L1 validator node based on zkEVM, placing the *ZeroNethermind* prototype in the context of the official roadmap.

2.5.1 The Blockchain Trilemma and Light Clients

Historically, the design of distributed ledgers has been dominated by the concept of the *Blockchain Trilemma*, a heuristic that postulates the impossibility of simultaneously maximizing scalability, security, and decentralization in symmetric validation architectures[23]. However, more recent academic literature has focused on the practical overcoming of these limitations through the adoption of zero-knowledge cryptographic proofs (ZKP)[3]. The introduction of zk-SNARK-based protocols radically alters the rules of engagement of the trilemma by introducing a strong computational asymmetry: in networks like Ethereum, security can be guaranteed no longer by the onerous redundant re-execution of all nodes, but by the lean mathematical verification of cryptographic proofs. Consequently, scalability is no longer a forced compromise against decentralization, but becomes an achievable engineering result by decoupling the complex production of proofs from their immediate verification.

This need for decoupling recalls the historical concept of the *Light Client*. As systematized in the document *SoK: Blockchain Light Clients*[4], light clients allow interaction with the blockchain without storing its entire history. While traditional light clients rely on trust assumptions, modern research is oriented toward using cryptographic accumulators and zk-SNARKs to allow devices with limited resources to

mathematically verify system integrity in a fully *trustless* mode.

2.5.2 The Verge and Ethereum's "Statelessness"

Overcoming the computational limitations of nodes is the core of the Ethereum roadmap development phase known as *The Verge*[2]. The primary objective is to democratize validation, making it so efficient that it can be performed on consumer devices (smartphones or smartwatches).

Currently, an Ethereum node requires storing a continuously growing state database, creating an I/O bottleneck that raises entry barriers and pushes toward centralization. The *Stateless Ethereum* paradigm[24] solves this problem by transforming blocks into self-contained execution units: instead of querying a local database, the stateless node receives a block accompanied by a *witness* and a cryptographic proof attesting to its validity.

In support of this profound structural transition, EIP-4762 (*Statelessness gas cost changes*)[8] has been proposed. This proposal introduces a remodulation of gas costs to accurately reflect the cryptographic burden of witness generation, disincentivizing state operations that would make cryptographic proofs too heavy or complex to generate.

2.5.3 The Layer 2 Landscape and Risk Analysis

Until now, the burden of scalability has been delegated to Layer 2s (L2s) through the *rollup-centric roadmap*. Analytical platforms such as L2BEAT[18] play a crucial role in evaluating the security trade-offs of these architectures, demonstrating how current rollups sit on a "trust spectrum." Many solutions still operate in *Stage 1*, relying on centralized sequencers, bridges controlled by multisigs, and *Security Councils* authorized to force state updates.

To mitigate these centralization vectors and risks arising from a frag-

mented ecosystem, research is shifting toward native integration of ZK proofs in Layer 1 (*Enshrined zkEVM*). This move aims to unify the security guarantees of the ecosystem, demonstrating that zkVM technology, matured on Layer 2, is ready for the base protocol layer.

2.5.4 Lean Ethereum: The Paradigm Shift

The paradigm shift of Lean Ethereum introduced by Justin Drake, which moves from the *"Trust but Verify"* model to the *"Verify the Math"* model, is outlined in the L1-zkEVM Roadmap 2026[14]. This document defines the end-to-end flow starting from an *Execution Client* (which generates the *ExecutionWitness*), passing through a zkVM guest program and a Prover, and ending at the validator node that asserts its validity.

In this direction, EIP-8025 (*Optional Execution Proofs*)[10] represents a fundamental step: it introduces an optional consensus-level modification that allows Beacon nodes to verify the validity of execution payloads through ZK proofs, without having to query a local execution client.

2.5.5 The zkVM Ecosystem: Implementations and Competition

The practical implementation of *real-time proving* rests on zero-knowledge virtual machines (zkVMs) capable of interpreting instruction sets, actively monitored by portals such as *zkevm.fyi*[7]. Among the leading projects are:

- **SP1**[25] (Succinct Labs): A high-performance *general-purpose* zkVM based on the RISC-V architecture, which allows generating proofs for programs compiled in Rust (or via LLVM).
- **ZisK**[1] (Polygon): A highly specialized zkVM architecture, designed specifically for rapid execution and proof generation for

EVM-compatible transactions.

The coexistence and competition among these *provers* are vital to ensure *client diversity* at the cryptographic level as well, preventing single points of failure (e.g., a bug in a ZK circuit) by requiring *multi-proofs* for block acceptance.

2.5.6 Prototypes and L1 Strawmap

The validity of this approach was demonstrated in the field during industry conferences such as Devconnect. On that occasion, zkLighthouse[5] was presented, a modified version of the Lighthouse consensus client. Operating on consumer-grade hardware, this node completely bypassed the use of a classic *Execution Client* (such as Geth or Nethermind), validating Mainnet blocks in real time. The system merely listened to a dedicated P2P channel, verifying in milliseconds the proofs produced by external prover clusters.

The entire architectural effort ultimately condenses into the L1 Strawmap (*strawmap.org*)[13], a draft roadmap maintained by the Ethereum Foundation team. This map unifies upgrades into three tracks (Consensus, Data, Execution) and sets protocol milestones, including the transition to a *Gigagas L1* (10,000 TPS) made possible precisely by the integration of zkEVM proofs and *real-time proving*.

The development of this thesis work is situated precisely at the intersection of these technologies, proposing the re-engineering of a *stateless Execution Layer* capable of acting as a pure proof validator within the *Fort Mode* framework outlined by the future of the Ethereum protocol.

Chapter 3

ZeroNethermind

This chapter presents the high-level design of ZeroNethermind, a Proof-of-Concept developed to demonstrate the feasibility of an Execution Layer client operating in a fully *stateless* mode through the verification of Zero-Knowledge cryptographic proofs.

The exposition is structured to illustrate the radical change in approach compared to traditional execution. Initially, the theoretical concept underlying the system, defined as the "Double Zero" paradigm, will be introduced. Subsequently, the three-layer architecture of the new validator will be analyzed, followed by a detailed description of the modified data flow within the Engine API. Low-level logic and code interoperability will be covered in `Chapter 4`, while the experimental performance validation is discussed in `Chapter 5`.

3.1 The "Double Zero" Paradigm

The design of ZeroNethermind rests on an architectural approach synthesized in the **"Double Zero"** paradigm, which combines two complementary mechanisms aimed at eliminating the node's bottlenecks:

- **Zero State**: the client does not maintain any local state database. Operating in `MemDb` mode, all persistence is ephemeral and lim-

ited to the headers of validated blocks only. The disk footprint is therefore close to zero.

- **Zero Knowledge**: block validity is not asserted through re-execution of transactions in the EVM, but through the cryptographic verification of ZKPs generated by external provers.

The adoption of this paradigm entails a radical shift of complexity: from native re-execution with cost $O(N_{gas})$, proportional to the volume of gas consumed in the block, to cryptographic verification with cost $O(1)$, independent of block complexity. In this model, a block containing a single transaction requires the same verification time as a block containing thousands of complex transactions.

3.2 Three-Layer Architecture

The architecture of ZeroNethermind is organized into three layers with distinct responsibilities, each isolated and communicating through well-defined interfaces. This stratification reflects both the engineering choices and the technological requirements (cross-language interoperability between C# and Rust).

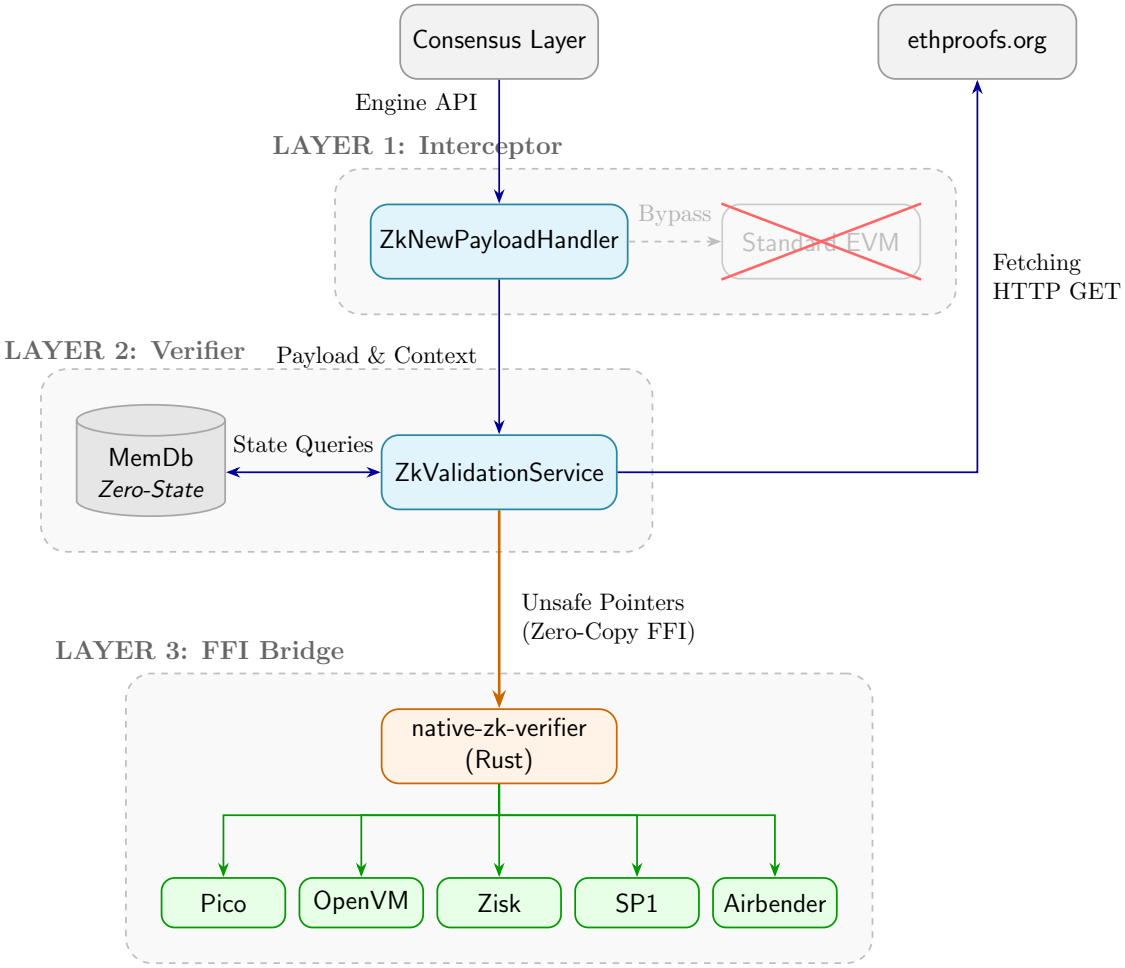

Figure 3.1: Core architecture of ZeroNethermind

Figure 3.1 illustrates the logical architecture and execution flow of the system, decomposed into three macro-levels:

3.2.1 Layer 1 (Interceptor)

Represents the point of contact with the Engine API. The custom handler (`ZkNewPayloadHandler`) intercepts incoming payloads from the Consensus Layer, fully bypassing the traditional state machine (`Standard EVM`) and radically reducing the native computational burden.

3.2.2 Layer 2 (Verifier)

Constitutes the main orchestration module, implemented in `C#`. The orchestrator service coordinates the parallel retrieval of proofs through HTTP calls to the external oracle (`ethproofs.org`). Additionally, it manages the ephemeral chain persistence: validated *headers* are saved in `MemDb`, allowing the client to operate in a state of total dependence on RAM (*Zero-State*), without resorting to any on-disk database.

3.2.3 Layer 3 (FFI Bridge)

Represents the high-performance interoperability boundary between the .NET *managed* environment and the native cryptographic libraries. To avoid bottlenecks caused by serialization and the Garbage Collector, binary payloads (proofs and verification keys) are passed to the `Rust` layer through pinned memory pointers (*Zero-Copy FFI*). This layer acts as a dispatcher toward the specific verification algorithms of individual zkVMs (Zisk, OpenVM, Pico, SP1, Airbender).

3.2.4 Ethproofs.org - External Oracle

In the ZeroNethermind architecture, ethproofs.org serves as the *single source of truth* for ZK proofs, thus eliminating the need for the PoC to operate its own proving infrastructure, which would require high-end GPU clusters and specialized expertise in proof generation. ZeroNethermind focuses exclusively on verification, entirely delegating generation to the external ecosystem.

3.3 The New Validation Flow

The validation flow in ZeroNethermind differs radically from the standard one described in section 2.1.6. The process follows these architectural steps:

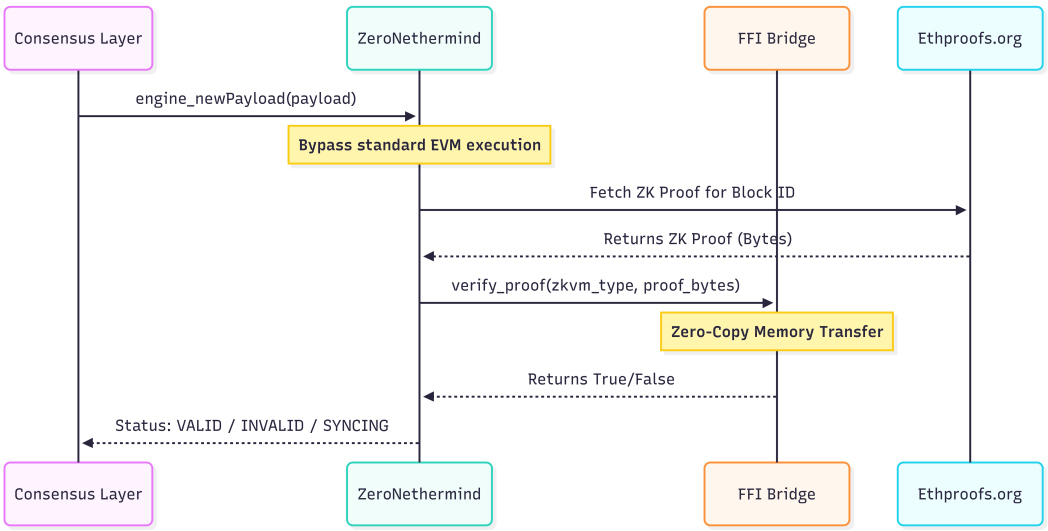

Figure 3.2: ZeroNethermind Validation Flow

1. **Block reception:** the CL receives a block from the gossip network and invokes `engine_newPayload` toward ZeroNethermind, transmitting the `ExecutionPayload`.
2. **Minimal pre-processing:** ZeroNethermind decodes the block and validates the header hash. Unlike the standard flow, checks that depend on the local state database (parent header verification, pre-pivot check, etc.) are not performed.
3. **Proof retrieval:** the system queries the ethproofs.org API to obtain the metadata of available proofs for the current block number.
4. **Cryptographic verification:** for each available proof, ZeroNethermind downloads the binary bytes of the proof and the verification key, then invokes the native verifier (Rust) via FFI.
5. **Multi-prover majority:** the block is considered valid if at least 50% of the verified proofs turn out to be valid. This approach mitigates the risk of bugs in a single zkVM circuit.
6. **Response to the CL:** ZeroNethermind returns the status `VALID`,

`INVALID`, or `SYNCING` to the Consensus Layer.

7. **Chain update:** upon receiving `engine_forkchoiceUpdated`, the system updates the canonical head.

It is important to emphasize that the node operates with a dedicated configuration profile (`zk-mainnet`) that explicitly disables unnecessary components — P2P, synchronization, TxPool — and activates MemDb mode. The details of this optimization will be further explored in section 4.4.

Table 3.1 synthesizes the architectural differences between a standard Nethermind node and ZeroNethermind. The comparison highlights how the PoC reverses the required resource profile: from the massive storage and intensive I/O of a full node, it shifts to a model dominated by network latency.

Characteristic	**Standard Nethermind**	**ZeroNethermind**
Input	Block + Full State	Block + ZK Proof
Action	Tx Re-execution (EVM)	Local Verification
State Storage	∼2 TB (RocksDB)	∼0 GB (MemDb, headers only)
Resources	High CPU, High SSD I/O	Low CPU, High Network
Bottleneck	Disk Speed (IOPS)	Proof Availability (Latency)
Cold Start	Hours (Snap Sync)	Seconds

Table 3.1: Architectural comparison between Standard Nethermind and ZeroNethermind.

Chapter 4

Implementation

This chapter documents the engineering journey from the first experimentation to the realization of a functioning ZeroNethermind prototype. The implementation was conducted in three incremental phases, each built on the foundations of the previous one: the creation of a standalone ZK verifier (section 4.1), its integration into the Nethermind client (section 4.2), and the orchestration of the complete validation service (section 4.3). The chapter concludes with the analysis of the configuration profile that transforms the client into a stateless validator (section 4.4).

4.1 Phase 1: The Foundation

The objective of the first phase was to build a standalone multi-prover ZK verifier, named `EthProofValidator`, to demonstrate the feasibility of .NET $\leftrightarrow$ Rust interoperability, independently of the Nethermind client.

4.1.1 The ZK Verifier Ecosystem

As described in `Chapter 1`, the zkEVM ecosystem is composed of multiple independent teams, each of which has developed its own zkVM

and corresponding proving system. For ZeroNethermind, these verifiers are treated as *black boxes*: the PoC interfaces exclusively with their verification APIs, without intervening in the internal logic.

A relevant technical aspect is that all major proving frameworks are implemented in Rust, the dominant language for high-performance cryptographic computation. This creates a significant integration challenge with the Nethermind client, which is entirely based on C#/.NET: a reliable and efficient bridge between the two runtimes is necessary.

The five zkVMs currently integrated in the PoC are: Zisk (Polygon Hermez), OpenVM (OpenVM.org), Pico (Brevis), SP1 Hypercube (Succinct), and Airbender (zkSync/Matter Labs). Each team exposes a conceptually analogous verification interface — a function that accepts a proof and a verification key, and returns a boolean result — but with differences in serialization formats and internal dependencies. These differences necessitate the creation of a unified Rust `crate` (`native-zk-verifier`) that acts as a common adapter.

4.1.2 WASM vs Native FFI

To build the C# $\leftrightarrow$ Rust bridge, two architectural strategies were evaluated: loading the verifiers as WebAssembly (WASM) modules, as done by ethproofs.org for *in-browser* validation, via the `Wasmtime.NET` runtime, and direct invocation through native Foreign Function Interface (FFI) with `P/Invoke`. Table 4.1 summarizes the comparison.

Aspect	WASM	Native FFI (P/Invoke)
Performance	Good (VM overhead).	Maximum (direct CPU execution).
Portability	Natively cross-platform.	OS-specific compilation.
Safety	Sandboxed.	`unsafe`: a Rust error can crash the .NET process.
Complexity	Requires the `Wasmtime` NuGet package.	No external dependencies.

Table 4.1: Comparison between WASM and Native FFI bridge strategies.

The choice fell on the native FFI approach due to the priority assigned to maximum performance. In a context where proof verification must occur within the 12-second window of an Ethereum slot, every overhead is significant. The WASM approach, while offering better isolation, introduces a level of indirection (the WebAssembly VM) that penalizes throughput in intensive verification scenarios. In exchange for the loss of sandboxing, the native bridge offers zero marshalling overhead and direct execution of instructions on the node's CPU.

4.1.3 The FFI Bridge: `native-zk-verifier`

The bridge between the two runtimes is realized by the Rust crate `native-zk-verifier`, configured as a `cdylib` (C-compatible shared library) and compiled with Rust nightly, as required by the cryptographic dependencies.

Rust Side

The crate exposes a single C-ABI function that acts as a dispatcher to the appropriate verifier:

```rust
#[no_mangle]
pub extern "C" fn verify(
    zk_type: u32,
    proof_ptr: *const u8, proof_len: usize,
    vk_ptr: *const u8,    vk_len: usize
) -> i32
```

Listing 4.1: FFI Dispatcher in Rust

The `zk_type` parameter selects the verifier via an integer enum. The entire call is wrapped in `panic::catch_unwind`, a Rust construct that catches any panics in the verifier code preventing them from propagating to the .NET host process, which would crash irreversibly. Return values follow a simple convention: `1` (Valid), `0` (Invalid), `-1` (Error).

Internally, each verifier module (`zisk.rs`, `openvm.rs`, `pico.rs`, `sp1_hypercube.rs`, `airbender.rs`) implements a common trait `Verifier` with a function `verify(proof, vk) -> Result<bool>`, abstracting the differences in serialization and underlying libraries.

C# Side

On the .NET side, the P/Invoke declaration uses the source-generated marshalling of .NET 8+:

```csharp
[LibraryImport("native_zk_verifier")]
public static partial int verify(
    int zk_type,
    byte[] proof, nuint proof_len,
    nint vk_ptr,  nuint vk_len);
```

Listing 4.2: P/Invoke Declaration in C#

The choice of `LibraryImport` over the more traditional `DllImport` eliminates runtime marshalling overhead, generating the interoperability code statically at compile time.

Memory Management

The memory ownership model between the two runtimes is a critical aspect of the bridge:

- **Verification Key (VK):** Allocated in unmanaged memory via `Marshal.AllocHGlobal`, outside the .NET Garbage Collector. This prevents the GC from moving or freeing the VK during the FFI call. The `ZkProofVerifier` wrapper implements `IDisposable` for deterministic release of unmanaged memory.
- **Proof data:** `Owned` by the C# GC and passed as a `byte[]` array. The .NET runtime performs automatic `pinning` during the P/Invoke call.
- **Rust:** Acts as a purely functional `guest`: it receives read-only pointers, performs the verification, and returns an integer. No memory allocation or deallocation occurs on the Rust side.

Automated Build

The integration between the two build systems is managed by a custom MSBuild target (`BuildRustDependency`) in the `.csproj` file:

1. Executes `cargo build --release` in the `native-zk-verifier/` directory.
2. Copies the resulting native library (`.so`/`.dll`/`.dylib`) to the .NET project output directory.

This ensures that a single `dotnet build` command automatically compiles both the C# and Rust code, significantly simplifying the development workflow.

4.1.4 The Multi-Verifier Console Application

With the FFI bridge operational, the first phase culminates in the creation of `EthProofValidator`: a standalone C# console applica-

tion that validates real mainnet blocks without any Nethermind client overhead.

Architecture

The application is structured into four components:

- `Program.cs`: CLI entry point that accepts `<LatestBlockId> <BlockCount>` and iterates over blocks invoking the validator.
- `BlockValidator`: Orchestrator that for each block retrieves proofs from ethproofs.org, distributes them to verifiers in parallel (`Task.WhenAll`), and applies the majority voting logic.
- `VerifierRegistry`: Registry pattern implemented with `ConcurrentDictionary<clusterId, ZkProofVerifier>`. Verifiers are created *lazily* on the first encounter with a new cluster, downloading the corresponding VK from the ethproofs.org API.
- `EthProofsApiClient`: HTTP client for the ethproofs.org REST APIs (fetch proofs, download binaries, download VK).

Majority Voting

The block acceptance logic follows a *majority voting* multi-prover algorithm: a block is considered valid if at least 50% of the non-skipped proofs are valid (`validCount * 2 >= totalCount`). This threshold represents a compromise between security and fault tolerance: a single verifier with errors does not invalidate the entire block, as long as the majority converges on the same result.

Standalone Demo Results

The application was executed on a sequence of 50 consecutive Ethereum mainnet blocks (#24199951–#24200000), with 5 proofs from 5 distinct zkVMs per block. Key results:

- **All 50 blocks validated successfully** (5/5 valid proofs per block).
- **Verification times per individual proof:** ∼5–50 ms (Zisk, Airbender), ∼90–140 ms (OpenVM, Pico).
- **Total time per block:** ∼1.6–2.5 s, dominated by HTTP download of proofs rather than cryptographic verification.

Below is an excerpt of the execution logs illustrating the processing of a subsequence of 3 blocks:

```
Processing Block #24199996
   Proof 5218300 - Zisk             : Valid (43 ms)
   Proof 5218298 - Zisk             : Valid (17 ms)
   Proof 5218303 - OpenVM           : Valid (114 ms)
   Proof 5218306 - Airbender        : Valid (13 ms)
   Proof 5218301 - Pico             : Valid (89 ms)
   --------------------------------
BLOCK #24199996 ACCEPTED (5/5)
Total Time: 2059 ms

Processing Block #24199997
   Proof 5218316 - Zisk             : Valid (40 ms)
   Proof 5218315 - Zisk             : Valid (15 ms)
   Proof 5218322 - OpenVM           : Valid (107 ms)
   Proof 5218326 - Airbender        : Valid (12 ms)
   Proof 5218319 - Pico             : Valid (112 ms)
   --------------------------------
BLOCK #24199997 ACCEPTED (5/5)
Total Time: 1868 ms

Processing Block #24199998
   Proof 5218337 - Zisk             : Valid (35 ms)
   Proof 5218340 - OpenVM           : Valid (111 ms)
   Proof 5218335 - Zisk             : Valid (6 ms)
   Proof 5218346 - Airbender        : Valid (33 ms)
   Proof 5218339 - Pico             : Valid (115 ms)
   --------------------------------
BLOCK #24199998 ACCEPTED (5/5)
Total Time: 1857 ms
```

Listing 4.3: Execution log excerpt from EthProofValidator

These results demonstrate the feasibility of the multi-verifier model and confirm that cryptographic verification is negligible compared to network latency, empirically validating the choice of the native FFI bridge.

4.2 Phase 2: Nethermind Integration

With the standalone verifier operational, the second phase addresses the integration of ZK validation logic into the Nethermind client lifecycle, without performing a fork of the upstream repository.

4.2.1 Alternatives Considered

Before arriving at the final solution, several integration strategies were evaluated and discarded:

1. **Strategy Pattern in the `NewPayloadHandler`:** Define an `IPayloadExecutionStrategy` interface with two implementations (`LocalExecutionStrategy` for the standard EVM, `ZkValidationStrategy` for ZK verification). Discarded because it required modifying the core standard handler code, violating the non-invasiveness principle.
2. **Modification of the `BlockchainProcessor`:** As suggested by the Nethermind team, intervening at the block processor level to bypass EVM execution. Discarded because the `BlockchainProcessor` is tightly coupled to the standard state machine and would have required excessively invasive modifications.
3. **Direct override of the `MergePlugin`:** Extending the existing plugin that manages post-Merge consensus. Discarded because it would have created a fragile dependency on the internal structure of the `MergePlugin`, subject to breakage with every client update.

4.2.2 The Plugin Architecture

The adopted solution leverages Nethermind's modular plugin system, based on the **Autofac** Dependency Injection container. The key interface is `IConsensusWrapperPlugin`, which allows:

- **Registering custom types** in the DI container via an `Autofac.Module`.
- **Overriding the handlers** of the Engine API without modifying Nethermind's core code.

This approach is the same one used by the `MergePlugin` (which manages post-Merge consensus) and the `ShutterPlugin` (which extends the consensus logic for the Shutter protocol), ensuring architectural compatibility with the client's design.

4.2.3 `ZkValidationPlugin`: The Entry Point

The `ZkValidationPlugin` plugin implements `IConsensusWrapperPlugin` and is the entry point of the entire PoC. Its activation is conditional: the `ZkValidation.Enabled` flag is set to `true` in the configuration file, and the `Merge` module is active (a prerequisite for post-Merge Engine API operation).

The `ZkValidationPluginModule` (Autofac Module) registers all necessary services in the DI container:

- `ZkValidationService` (shared orchestrator)
- `ZkNewPayloadHandler`
- `ZkForkchoiceUpdatedHandler`
- `BlockValidator`
- `EthProofsApiClient`

4.2.4 Engine API Override

`ZkNewPayloadHandler`

The ZK handler for `engine_newPayload` replaces the standard `NewPayloadHandler` and radically simplifies pre-processing. Table 4.2

summarizes the retained and eliminated checks.

Check	ZK	Rationale
Block Decoding	✓	Necessary to obtain the data structure
Hash Validation	✓	Cryptographic integrity of the header
Invalid Chain Tracker	✓	Invalidity propagation
Pre-Pivot Check	×	Irrelevant without synchronization
Parent Header Lookup	×	No local database
Orphan Block Insertion	×	No sync in progress
ShouldProcessBlock Check	×	ZK validation is always desired
TTD Checks	×	Post-merge blocks only
EVM Execution	×	**Replaced by ZK verification**

Table 4.2: Pre-processing check comparison: standard handler vs. ZK handler.

After the simplified pre-processing, the handler delegates validation to the `ZkValidationService`, which verifies the availability of proofs within a maximum time, downloads them, and performs the validation locally, returning `VALID` or `INVALID` accordingly. If proofs are not available, the EL returns the `SYNCING` status, waiting again for the `engine_newPayload` call from the CL.

`ZkForkchoiceUpdatedHandler`

The ZK handler for `engine_forkchoiceUpdated` bypasses the `WasProcessed` check, a flag normally set to `true` only after EVM execution and therefore always `false` in ZK mode. The residual operations are:

1. Verify that the block does not belong to an invalidated chain (`IsOnInvalidChain`).

2. Look up the block in the `blockTree`.
3. Update the `blockTree.ForkChoiceUpdated(...)`.
4. Return `VALID` with the updated `headBlockHash`.

4.3 Phase 3: The ZK Validation Service

The third phase integrates the verification logic into the complete operational context of the client, addressing the challenges of prover latency, cache management, and interaction with the consensus protocol.

4.3.1 `ZkValidationService`: The Orchestrator

The `ZkValidationService` is the shared service between the two Engine API handlers. Its responsibilities include:

- **Validated block cache:** Before each verification, the service checks whether the block is already present in the `blockTree` via `FindBlock(hash)`. If so, it immediately returns `VALID` without re-verification, avoiding redundant work when the CL sends the same block both via `newPayload` and via `forkchoiceUpdated`.
- **BlockTree insertion:** On a `VALID` result, the block is inserted with `blockTree.Insert(block, SaveHeader, BeaconBlockInsert)`, persisting only the header in `MemDb` without writing any state data.
- **Invalidity handling:** On an `INVALID` result, the service invokes `invalidChainTracker.OnInvalidBlock(hash, parentHash)`, marking the entire descendant chain as invalid and propagating the information to subsequent child blocks.

4.3.2 Managing the SYNCING Protocol

The main engineering challenge of Demo 1 (section 5.1.2) was managing prover latency. When a newly produced block from the network reaches the node, the cryptographic proof might not yet be available on ethproofs.org (the average proving time is approximately 5–15 seconds). Without an adequate management mechanism, the node would enter a permanent `SYNCING` state, as the CL would continue sending new blocks before previous ones were verified.

The implemented solution is an **Active-Wait pattern**: the `ZkValidationService` executes a configurable retry loop that blocks the RPC thread while waiting for the proof. The operational flow is as follows:

1. The `BlockValidator` queries ethproofs.org and finds no available proofs → returns `SYNCING`.
2. The service waits `RetryDelayMs` and retries the query up to the maximum number of attempts.
3. The CL (Lighthouse) receives `SYNCING` and switches to "optimistic" mode (provisionally assumes the block is valid while awaiting EL confirmation).
4. Once the proof is available and verified, the service returns `VALID`.
5. Lighthouse transitions to `INFO Synced ... (verified)`.

This approach represents a **functional workaround** for the PoC: blocking the RPC thread is acceptable in a demonstration context, but would not be sustainable in a production environment where Engine API responsiveness is critical. A production-ready implementation requires prior adaptation and modification of the current protocol.

4.3.3 BlockValidator Adaptation

The `BlockValidator` developed in section 4.1 (standalone) was adapted to the Nethermind plugin context with the following modifications:

- Integration with Nethermind's logging system (`ILogManager`) for unified diagnostics.
- Sharing of the `VerifierRegistry` and `EthProofsApiClient` through the Autofac DI container, rather than direct instantiation.
- Same majority voting logic, with adapted error handling: an error on a single verifier does not interrupt the process, but is logged and counted in the vote.

4.4 Configuration Profile

The `zk-mainnet.json` file defines the operational profile that transforms Nethermind into a "Fort Mode" validator. Table 4.3 summarizes the disabled components and their respective rationale.

Namespace	Parameter	Value	Rationale
Init	DiagnosticMode	MemDb	All stores in memory, zero disk I/O
Init	DiscoveryEnabled	false	No P2P peer discovery
Init	PeerManagerEnabled	false	No P2P connection management
Init	ProcessingEnabled	false	Disables the `BlockchainProcessor`
Sync	SynchronizationEnabled	false	No synchronization
TxPool	Size	0	No transaction gossip, no mempool

Table 4.3: Modules disabled in the `zk-mainnet` profile.

The modules that remain active are exclusively those necessary for stateless validator operation: `JsonRpc` (with only the `Net`, `Web3`, `Engine` modules for CL communication), `Merge` (prerequisite for post-Merge Engine API), `ZkValidation` (plugin activation), `HealthChecks`, and `Metrics` (monitoring and telemetry).

The result is a node that has no on-disk database (MemDb instead of the ∼2 TB of RocksDB), does not discover or manage peers (receives blocks exclusively from the adjacent CL), does not execute transactions (the
`BlockchainProcessor` is deactivated), does not participate in transaction gossip, and validates exclusively through ZK proofs. This profile concretely materializes the "Fort Mode" concept described in `Chapter 1`: a lightweight validator, deployable on consumer hardware, that contributes to network security by delegating proof generation to the "Beasts" and focusing solely on cryptographic verification.

Chapter 5

Validation

The validation of the ZeroNethermind PoC is articulated around two complementary research questions:

1. **Operational feasibility:** Can a stateless verifier maintain synchronization with the L1 Mainnet chain in real time, operating exclusively through cryptographic proofs?
2. **Resource impact:** What are the quantitative differences in terms of verification speed, CPU consumption, RAM, disk I/O, and storage between a standard full re-execution node and a ZeroNethermind node?

To answer these questions, two distinct experimental campaigns were designed: **Demo 1** (Section 5.1.2), which performs a live validation on the Ethereum Mainnet through proofs provided by `ethproofs.org`, and **Demo 2** (Section 5.1.3), which uses a controlled environment based on Kurtosis[17] to isolate and compare hardware metrics under deterministic conditions.

5.1 Methodology

5.1.1 Test Environment

All experiments were executed on the same physical machine, using Docker containers to ensure isolation and reproducibility. The hardware specifications are reported in Table 5.1.

Component	Specification
Operating System	Manjaro Linux, kernel 6.12.73 (64-bit)
CPU	4 x Intel Core i5-7400 @ 3.00 GHz
RAM	16 GiB DDR4
GPU	NVIDIA GeForce GTX 1060 6 GB
Network	Home connection (FTTC)
Containerization	Docker Engine with `docker-compose`

Table 5.1: Hardware specifications of the test machine.

The choice of mid-range consumer hardware is intentional: the PoC's objective is to demonstrate that ZK validation is achievable on common devices, without the need for specialized servers.

5.1.2 Demo 1: Real-Time Validation on the Mainnet

Setup

Demo 1 consists of deploying ZeroNethermind paired with a Lighthouse Consensus Layer, both running as Docker containers on the local machine.

- **Execution Layer:** ZeroNethermind with the `zk-mainnet` profile (section 4.4), configured in `MemDb` mode (zero disk storage), with P2P discovery, peer manager, synchronization, and `BlockchainProcessor` disabled.

- **Consensus Layer:** Lighthouse in `checkpoint-sync` mode, synchronized to the Beacon Chain tip via the public checkpoint `mainnet.checkpoint.sigp.io`.
- **Proof source:** `ethproofs.org` via HTTP API, automatically queried by the `BlockValidator` for each block received via `engine_newPayload`.
- **Monitoring:** Prometheus[22] + cAdvisor[15] for container metrics sampling (CPU, RAM, I/O, bandwidth) at 15-second intervals.

Objective

Demonstrate the operational feasibility of the "Fort Mode" paradigm: the node, devoid of a database and EVM execution capability, must successfully validate Mainnet blocks in real time, maintaining synchronization with the CL and responding to each `engine_newPayload` call with a `VALID` status within the slot's time constraints.

Procedure

The test was conducted over a window of approximately 17 minutes, during which the node received and validated **121 Mainnet blocks** (in the range `#24628200` – `#24628397`), corresponding to approximately 3 Beacon Chain epochs. For each block, the `BlockValidator`:

1. Queried `ethproofs.org` to obtain the list of available proofs.
2. Downloaded in parallel the binary proofs generated by different independent zkVMs: **Zisk** (typically 2 instances), **OpenVM**, **Pico**, and **Airbender**.
3. Invoked the native FFI bridge (`libnative_zk_verifier.so`) for the cryptographic verification of each proof.
4. Performed majority voting: the block is accepted if at least 50% of the verified proofs are valid.

5. Inserted the block (*header-only*) into the in-memory `BlockTree` and returned `VALID` to the CL.

For each validated block, the system recorded in the log: the total validation time, the maximum download time (bottleneck of HTTP parallelism), the maximum FFI verification time (bottleneck of cryptographic parallelism), and the valid/total proof count. Data was extracted from container logs and aggregated in CSV format.

5.1.3 Demo 2: Controlled Environment (Kurtosis Lab)

Setup

Demo 2 uses a Kurtosis enclave to create a completely isolated local Ethereum devnet, configured to directly compare the hardware footprint of a standard Nethermind node and a ZeroNethermind node.

- **Standard nodes:** 2 Nethermind instances with default configuration, equipped with validators and MEV-Boost.
- **ZeroNethermind node:** 1 instance with the `zk-mainnet` profile and the `ZkValidation` plugin active, connected to the local **Mock Prover**.
- **Consensus Layer:** Lighthouse for all nodes.
- **Mock Prover:** A containerized service (`zero-prover`) that replicates the `ethproofs.org` API. For each block it returns **5 instances** of the same pre-generated, valid Airbender cryptographic proof, simulating the real workload in which the node downloads and verifies proofs from 5 independent zkVMs in parallel. This choice allows measuring FFI verification time under realistic conditions, while simultaneously eliminating the variability of network latency toward an external service.
- **Workload:** `spamoor` generates a continuous flow of transactions to saturate blocks at the current Gas Limit.

- **Monitoring:** Prometheus, Grafana, and `ethereum_metrics_exporter` for integrated metrics collection of all nodes.

The use of MEV-Boost for both node types is intentional: by delegating block construction to an external builder, all nodes are limited to the validation phase only, which is exactly the component under test.

Objective

Quantify the hardware resource gap between validation via EVM re-execution ($O(N_{gas})$) and validation via cryptographic verification ($O(1)$), in an environment where network conditions, transactional load, and hardware are identical for all nodes.

Procedure

Demo 2 adopts a **progressive load** protocol lasting ∼45 minutes, articulated in two phases:

1. **Baseline Phase** (first ∼15 minutes): the devnet operates with minimal transactional throughput (1 transaction per spammer type), in order to establish reference values for all metrics of interest.
2. **Under-load Phase** (last ∼30 minutes): the `spamoor` throughput is manually increased in 4 successive steps, gradually increasing the number of transactions per type (`storagespam`, `gasburnertx`, `erc20tx`, `eoatx`) until blocks are saturated at the 500 M Gas Limit.

Throughout the entire experiment, metrics are collected through two complementary sources: (i) Nethermind's native metrics exposed via Prometheus (particularly `nethermind_new_payload_execution_time` for the `newPayload` validation time), and (ii) cAdvisor for container-level resource metrics (CPU, RAM, I/O, bandwidth, storage).

The results of Demo 2 are presented in section 5.3.

5.2 Demo 1: Validation on the Mainnet

The results of Demo 1 are presented through two complementary visualizations:

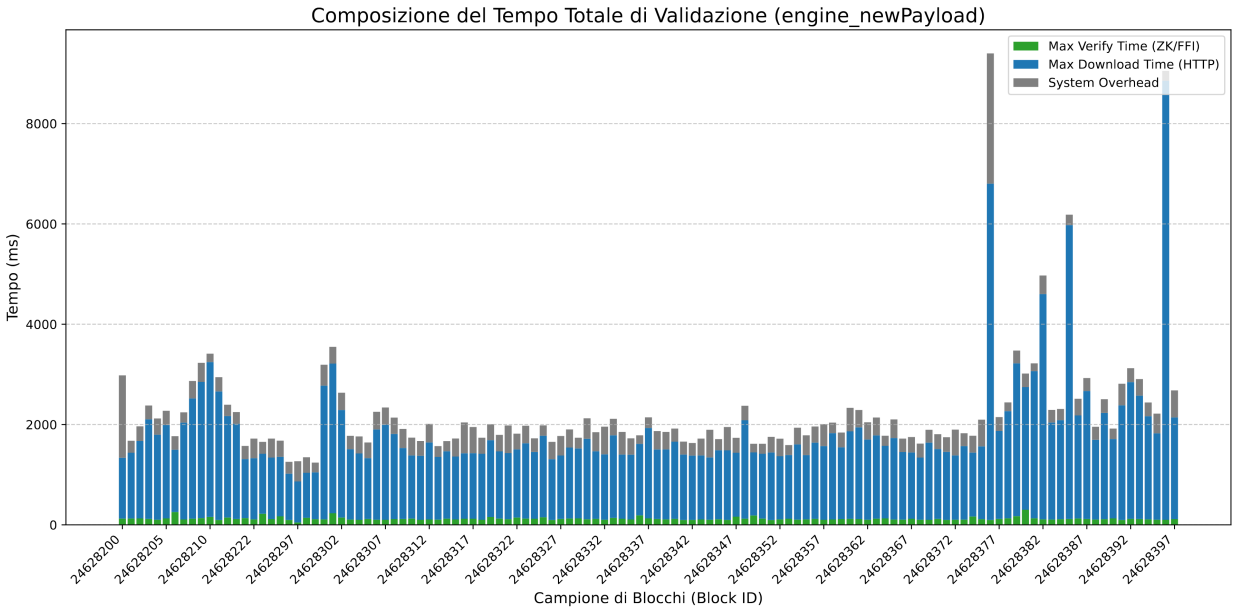

Figure 5.1: Composition of Total Validation Time

Graph 5.1 illustrates the decomposition of total time to validate each block within the `engine_newPayload` handler. As clearly evidenced by the predominance of the blue component over the green one, the *Max Download Time* (the time spent retrieving proofs via HTTP from ethproofs.org) constitutes the true and only bottleneck of the process, regularly absorbing over 90% of the total time. In contrast, the *Max Verify Time* (in green), i.e., the actual computation time spent by the CPU for cryptographic verification via the FFI bridge, remains constant and negligible (on the order of 100-150 ms), regardless of block complexity. The anomalous peaks visible (up to ∼9000 ms) are attributable exclusively to transient network latencies of the external provider.

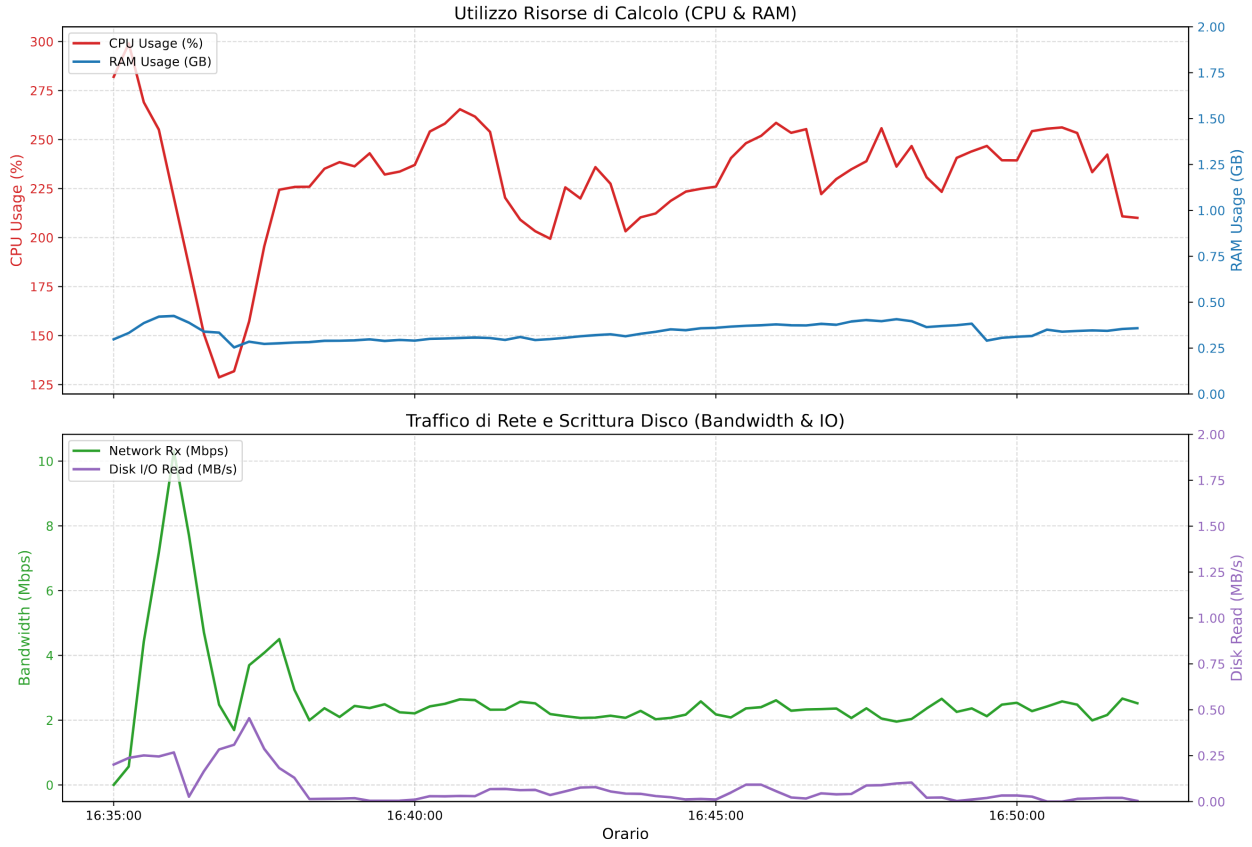

Figure 5.2: Performance Analysis: ZeroNethermind Stateless Node

Graph 5.2 empirically confirms the success of the "Zero State" paradigm through temporal analysis of hardware resource usage. In the upper panel, RAM usage (blue line) remains extraordinarily low and flat, consistently settling below 400 MB. This demonstrates the effectiveness of eliminating the persistent state database and the Mempool. In the lower panel, the most significant metric is disk I/O traffic (purple line), which, after minimal absorption during startup, drops to zero entirely (0 MB/s), confirming the total absence of the heavy read-/write operations on the MPT tree typical of the EVM. Concurrently, the incoming network bandwidth (green line) shows regular patterns associated with the continuous download of cryptographic proofs for each new slot, confirming that the system has shifted from an *I/O-*

bound limit to a *Bandwidth-bound* limit.

Table 5.2 reports the aggregated statistics of the session.

Metric	Mean	Median	Min	Max
Total validation time (ms)	2 237	1 950	1 241	9 400
Max download time (ms)	1 768	1 447	817	8 755
Max verify time (ms)	122	116	48	301
RAM container (MB)	360	335	272	457
CPU container (%, per-core)	57.7	—	32.2	74.8
Bandwidth RX (KB/s)	333	295	<1	1 289
I/O read (KB/s)	77	34	<1	476

Table 5.2: Aggregated statistics of Demo 1 (121 Mainnet blocks).

The most significant result is the disproportion between download time and verification time: the average FFI cryptographic verification time is only **122 ms** (median 116 ms), representing just **5.5%** of the average total validation time. The remaining 94.5% is dominated by network latency for downloading proofs from `ethproofs.org`. This finding confirms that ZK verification is computationally negligible compared to data transport, and anticipates a fundamental property: when proofs are included directly in the block payload, the validation time will be reduced exclusively to the verification component, on the order of ∼100 ms.

5.3 Demo 2: Analysis in Controlled Environment

The results of Demo 2 quantitatively confirm the hypotheses formulated in `Chapter 1`: the validation paradigm via cryptographic verification ($O(1)$) presents structural advantages over EVM re-execution ($O(N_{gas})$), both in terms of processing time and hardware resource consumption.

5.3.1 Verification Speed

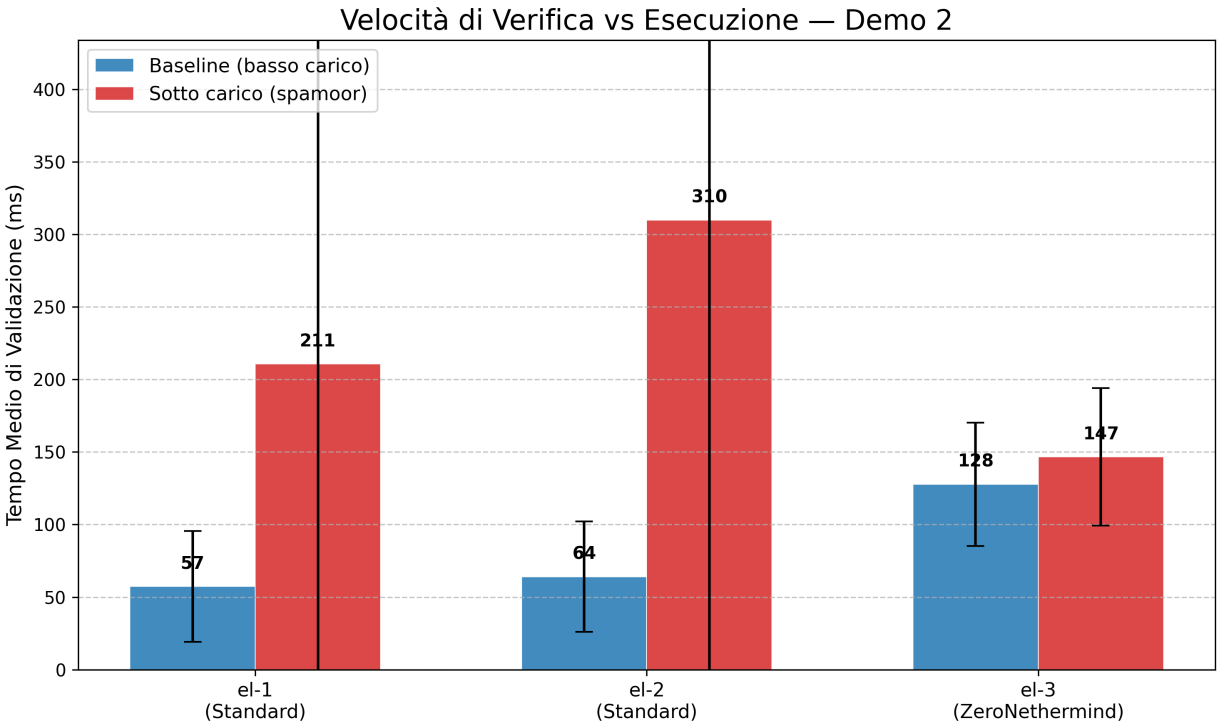

Figure 5.3: Verification Speed vs Execution: comparison of average validation times in the Baseline and under-load phases.

Graph 5.3 reveals a fundamental asymmetry between the two validation approaches. Table 5.3 reports detailed statistics for each node in the two experimental phases.

Node	Baseline		Under load		Overall	
	Mean	Max	Mean	Max	Mean	Max
el-1 Standard	57	264	211	1 129	162	1 129
el-2 Standard	64	203	310	1 487	226	1 487
el-3 ZeroNethermind	128	268	147	334	141	334

Table 5.3: `engine_newPayload` validation time (ms) per phase.

Under low-load conditions (*Baseline*), the standard nodes are faster (∼57–64 ms) compared to ZeroNethermind (∼128 ms), since re-execution

of lightweight transactions is computationally less expensive than downloading and verifying 5 cryptographic proofs in parallel. However, under increasing transactional load, the ratio reverses drastically: standard nodes slow down to ∼211–310 ms with peaks up to ∼1,500 ms, while ZeroNethermind remains essentially unchanged at ∼147 ms. The average increase for standard nodes is ∼ 3.7× relative to baseline, versus an increase of only 15% for ZeroNethermind. This is the empirical manifestation of $O(N_{gas})$ vs $O(1)$ complexity: cryptographic verification time is independent of the block's computational complexity.

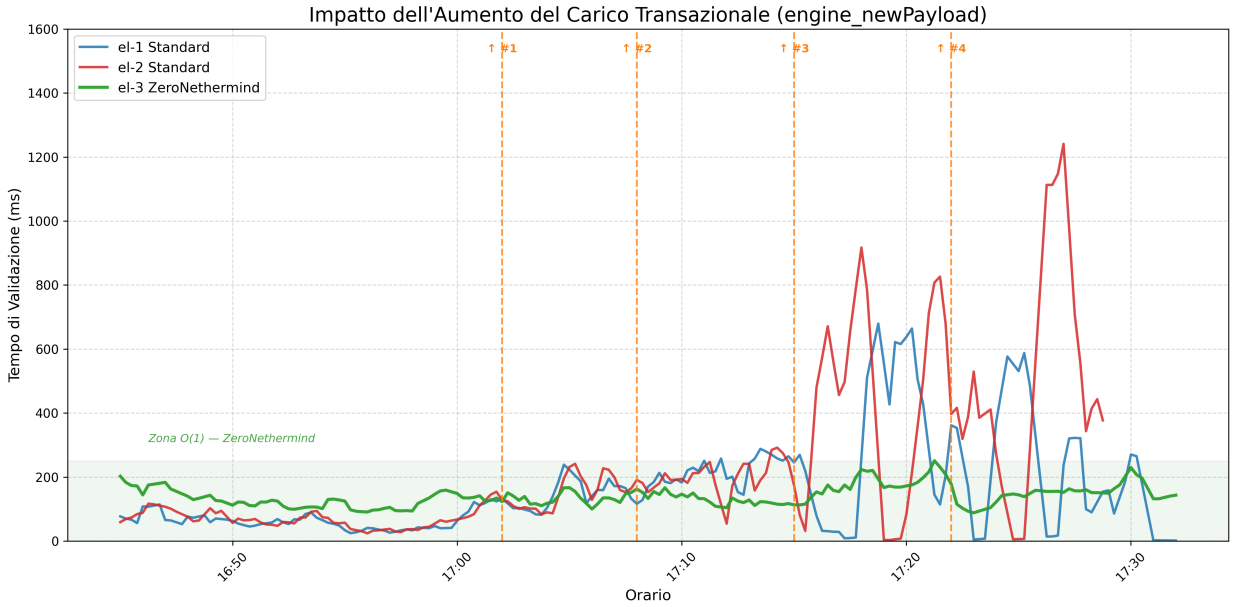

Figure 5.4: Impact of increasing transactional load on validation time (`engine_newPayload`).

Graph 5.4 provides the temporal dimension of the phenomenon. The 4 dashed vertical lines indicate the moments when spammer throughput was manually increased. To quantify the progressive divergence, Table 5.4 reports the averages for 5-minute windows.

Window	el-1 (ms)	el-3 (ms)	Ratio el-1/el-3
16:45–16:50	76	153	0.50×
16:50–16:55	60	116	0.52×
16:55–17:00	36	114	0.31×
17:00–17:05	**118**	**132**	**0.90**×
17:05–17:10	172	139	1.24×
17:10–17:15	228	124	1.84×
17:15–17:20	295	168	1.76×
17:20–17:25	293	152	1.93×

Table 5.4: Average validation time trend for 5-minute windows (UTC). The bold row indicates the crossover.

During the Baseline phase (first 3 windows), the standard node is up to 3× faster. The crossover manifests in the 17:00–17:05 UTC window, corresponding to the first throughput increase (↑#1), where the two approaches are equivalent (0.90×). From that point onward, the standard node becomes progressively slower: after ↑#3, el-1 takes nearly twice the time of el-3 (ratio 1.93×). The standard node curves (blue and red) diverge upward with increasing slope, a behavior consistent with the linear complexity of EVM re-execution. In contrast, the ZeroNethermind curve (green) remains flat within the ∼100–200 ms band regardless of load increases, confirming an empirically constant verification time, independent of the volume of gas consumed.

5.3.2 Resource Footprint

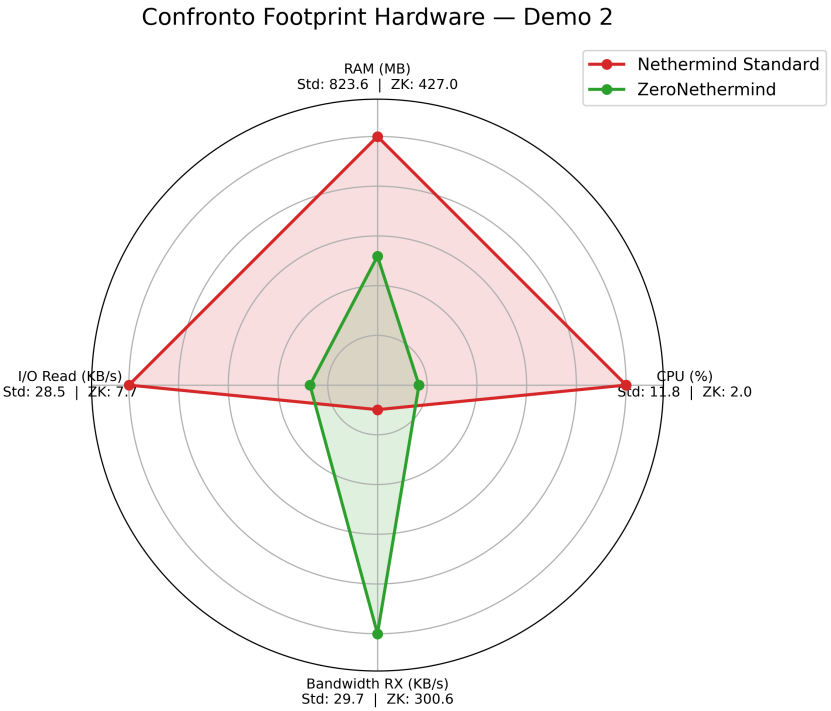

Figure 5.5: Hardware footprint comparison between Standard Nethermind (average of 2 nodes) and ZeroNethermind.

Graph 5.5 shows the resource profile of the two architectural approaches across 4 dimensions, normalized to the maximum value of each axis. Two complementary profiles emerge that reflect radically different validation architectures:

- **Standard Nethermind:** Dominates on CPU ($\sim$11.8% vs $\sim$2.0%, a factor of $\sim$6$\times$) and RAM ($\sim$823 MB vs $\sim$427 MB), reflecting the heavy computational burden of EVM re-execution and in-memory management of the Merkle Patricia Trie (MPT). A higher disk I/O Read is also noted ($\sim$28.5 KB/s vs $\sim$7.7 KB/s) necessary to

load the state branches required by transactions.

- **ZeroNethermind:** Reverses the paradigm, shifting the predominant load to incoming Bandwidth (∼300.6 KB/s vs ∼29.7 KB/s, a factor of ∼10×). Local resources (CPU, RAM, Disk) remain compressed near the center of the graph.

This radar chart visually quantifies a key architectural principle: the transition to cryptographic validation does not "magically" zero out the network's cost, but translates its nature, replacing expensive and rigidly limited local resources (CPU, disk IOPS) with an external, elastic, and naturally scalable resource (bandwidth).

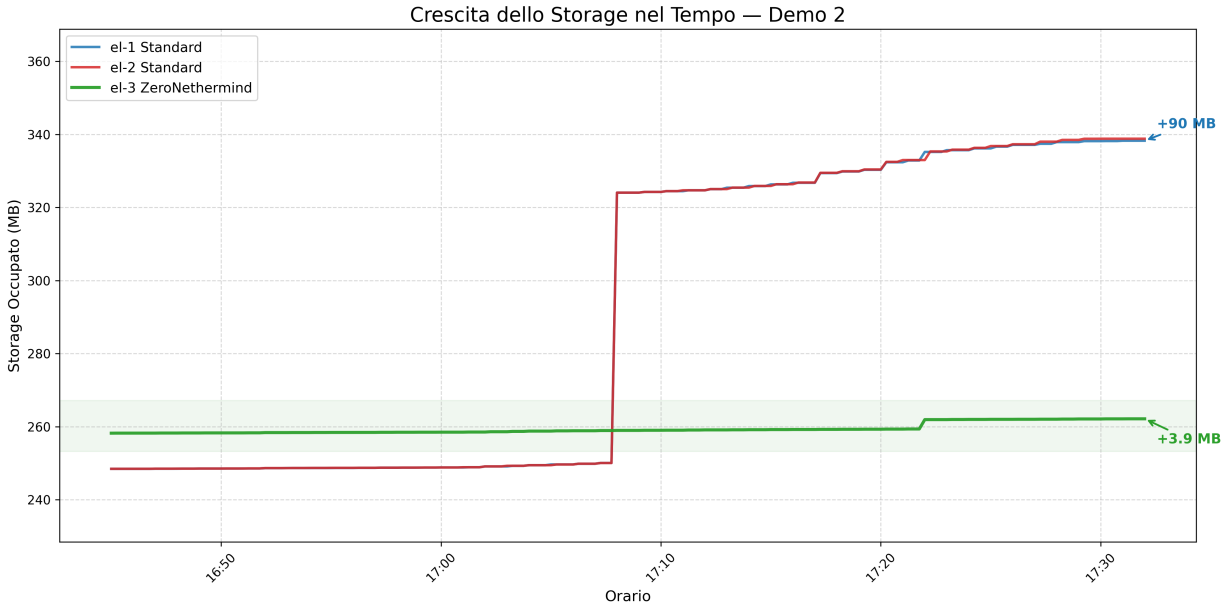

Figure 5.6: Storage growth over time: comparison of on-disk state accumulation.

Graph 5.6 empirically demonstrates the *Zero State* paradigm introduced in `Chapter 1`. The two standard nodes start at ∼248 MB and reach ∼338 MB in ∼45 minutes (growth of ∼90 MB each). ZeroNethermind, starting from ∼258 MB, reaches only ∼262 MB, equal to **4.3%** of the standard growth. The residual growth of ZeroNethermind is attributable solely to the client's operational metadata (internal logs,

Consensus Layer cache), not to blockchain state accumulation.

This result demonstrates that eliminating the persistent state database is not just a theoretical advantage, but produces a measurable and significant impact on the long-term operational sustainability of a validator node.

5.4 Discussion of Results

The results of the two experimental campaigns, although conducted in distinct operational contexts, converge toward a coherent picture that can be synthesized along three thematic axes.

1. Computational paradigm: from $O(N_{gas})$ to $O(1)$. Demo 1 demonstrated that FFI cryptographic verification requires on average ∼122 ms per Mainnet block, regardless of the block's complexity (up to 30M Gas). Demo 2 confirmed this property under controlled and increasing load: while standard nodes see validation time increase by ∼3.7× going from the Baseline to the under-load phase (with peaks up to ∼1,500 ms), ZeroNethermind remains in the 128–147 ms range, showing an increase of only 15%. The crossover between the two approaches occurs already with moderate transactional loads (Table 5.4), suggesting that the ZK verification advantage manifests *before* block saturation.

2. Shift of the architectural bottleneck. Both Demo 1 and Demo 2 confirm the shift of the bottleneck from *I/O-bound* to *bandwidth-bound*. In Demo 1, downloading proofs from `ethproofs.org` absorbs 94.5% of total validation time, relegating FFI verification to 5.5%. In Demo 2, the radar chart (Graph 5.5) quantifies this shift: ZeroNethermind's bandwidth consumption is ∼10× that of standard nodes, while CPU and RAM are reduced by ∼6× and ∼2× respectively. This result has an important design implication: once proofs are included di-

rectly in the block payload (as planned by Ethereum's roadmap), the bandwidth bottleneck will be eliminated, and the validation time will coincide with the FFI component alone ($\sim$120 ms).

3. Operational stability and State Bloat elimination. During Demo 1, ZeroNethermind validated 121 consecutive Mainnet blocks in synchronization with the Lighthouse Consensus Layer, never missing an attestation slot and maintaining RAM stably below 400 MB. Demo 2 extended this observation to $\sim$45 minutes of sustained load, confirming that memory and CPU remain flat even under stress. Perhaps the most relevant finding concerns storage: disk growth is 4.3% of the standard (Graph 5.6), demonstrating that the *Zero State* paradigm eliminates the *State Bloat* problem that constitutes one of the main obstacles to Ethereum's decentralization.

Table 5.5 synthesizes the architectural and performance differences emerged from the two campaigns between a Nethermind Full Node and the ZeroNethermind proof-of-concept.

Parameter	Nethermind Full Node	ZeroNethermind
Trust Model	Local re-execution N-of-N	Mathematical verification 1-of-N
State Storage	$\sim$1 TB (Continuous growth)	$\sim$0 GB (MemDb in RAM)
Required RAM	16–32 GB	$<$500 MB (Plateau)
CPU Impact	Multi-core saturation (EVM)	Minimal transient peaks ($\sim$120 ms)
Disk I/O (IOPS)	High (Random MPT access)	None
Bottleneck	Hardware Limits (I/O-bound)	Network Latency (Bandwidth-bound)
Complexity	$O(N_{gas})$ — Linear to Gas Limit	$O(1)$ — Cryptographic constant
Synchronization	Hours / Days (State Sync)	Seconds (Checkpoint Sync)
Extended Services	Full RPC, Block Building	Attestation Only (Fort Mode)

Table 5.5: Summary of the architectural leap between a Stateful node and a Stateless validator.

Chapter 6

Conclusion

This concluding chapter synthesizes the contributions of the work performed, analyzes its engineering limitations, outlines future development directions, and frames the proof-of-concept within the evolutionary trajectory of the Ethereum protocol.

6.1 Summary

This thesis has demonstrated, through the design, implementation, and experimental validation of the ZeroNethermind proof-of-concept, that an Ethereum Execution Layer client can operate in a fully *stateless* mode, relying on the mathematical verification of Zero-Knowledge cryptographic proofs rather than the computational re-execution of transactions.

The PoC introduced three main architectural innovations:

1. **The "Double Zero" paradigm:** The simultaneous elimination of the local state database (Zero State, via MemDb) and EVM re-execution (Zero Knowledge, via cryptographic verification), realizing for the first time an L1 validator that requires neither terabytes of storage nor multi-core computational power.
2. **The cross-language FFI bridge:** A high-performance inter-

operability interface between Nethermind's managed C#/.NET runtime and native Rust cryptographic libraries, capable of verifying proofs generated by 5 independent zkVMs (Zisk, OpenVM, Pico, SP1, Airbender) with Zero-Copy transfer of binary payloads.

3. **The plugin architecture:** Non-invasive integration into the Nethermind client through the plugin system, which allows intercepting and overriding the Engine API handlers without modifying the core code, preserving compatibility with future client updates.

The experimental validation, conducted on two complementary fronts, produced significant quantitative results:

- **Demo 1 (Mainnet)** validated 121 blocks in synchronization with the Lighthouse Consensus Layer. The average FFI cryptographic verification time was only **122 ms** (median 116 ms), representing just 5.5% of the total time, with the remaining 94.5% dominated by proof download. RAM remained stably below 400 MB and disk I/O dropped to zero after the startup phase.
- **Demo 2 (Kurtosis devnet)** empirically confirmed the $O(1)$ property of cryptographic verification under increasing load: while standard nodes saw validation time increase by $\sim$3.7$\times$ with peaks up to $\sim$1,500 ms, ZeroNethermind remained in the 128–147 ms range (increase of only 15%). CPU usage was $\sim$6$\times$ lower, RAM $\sim$2$\times$ lower, and disk growth was **4.3%** of the standard, demonstrating a 95.7% reduction in local state accumulation.

These results positively answer the two research questions posed during the design phase: (i) validation via cryptographic verification is *operationally feasible* within a production-ready client, operating within the 12-second slot time constraints; (ii) the transition from re-execution to ZK verification produces a *measurable and significant impact* on the resource profile, shifting the bottleneck from disk I/O

to network latency.

The native support for 5 independent zkVMs also constitutes an initial form of *multi-prover diversity*: should a single zkVM present a bug in its arithmetic circuits, the majority voting logic would distribute the risk across multiple independent implementations, offering structural mitigation against cryptographic vulnerabilities.

6.2 Limitations

Despite the encouraging results, the PoC presents three engineering limitations that prevent its direct adoption in a production environment.

1. The latency of the external HTTP oracle. ZeroNethermind depends on the REST APIs of `ethproofs.org` for retrieving cryptographic proofs. This dependency on a centralized HTTP oracle introduces unacceptable network latency for the protocol's timing constraints: in Demo 1, proof download absorbed 94.5% of total validation time, leaving the effective FFI verification with only ∼120 ms. In an ideal scenario where proofs were immediately available, the validation time would drop by an order of magnitude. Furthermore, the dependency on a single centralized endpoint represents a *single point of failure*, incompatible with the protocol's decentralization principles.

2. The `SYNCING` protocol workaround. To handle the case where the proof for a block is not yet available at the time `newPayload` is received, the PoC adopts an *Active-Wait* strategy: the custom handler responds to the CL with `SYNCING` status and blocks the RPC thread in a polling loop until the proof becomes available on ethproofs.org. This approach, while functional for demonstration purposes, is not scalable in production: thread blocking introduces non-deterministic latency

and could cause Consensus Layer timeouts in case of prolonged delays in proof generation.

3. The Data Availability gap. A stateless client, by definition, does not possess the blockchain's state database. This implies the impossibility of responding to standard user RPC queries, such as `eth_getBalance`, `eth_call`, or `eth_getStorageAt`, which require direct access to the current or historical state. The PoC is therefore strictly defined as an **attester-only** node ("Fort Mode"): it validates the chain with complete cryptographic guarantees, but cannot serve as a *full-service* RPC endpoint for decentralized applications or wallets. This is an intentional design choice that sacrifices extended RPC functionality in favor of extreme resource efficiency.

6.3 Future Work

The limitations described in the previous section are not intrinsic to the stateless validation paradigm, but reflect constraints of the current state of the ecosystem. Ethereum's roadmap envisions several evolutions that would address each of them.

1. ePBS and the expansion of the proving window. The current protocol grants validators approximately 4 seconds to receive and validate a block within the 12-second slot. The adoption of *Enshrined Proposer-Builder Separation* (ePBS, EIP-7732)[9], planned for the Glamsterdam hardfork (estimated for the second half of 2026), would expand the time window available for attesters from ∼2 seconds to ∼6–9 seconds[14]. This increase is crucial for the feasibility of real-time proving: with a 9-second window, even current provers could generate proofs within the slot's time constraints.

2. P2P proof gossip and protocol inclusion. The dependency on a centralized HTTP oracle would be eliminated through two progressive evolutions:

- **P2P Proof Gossip:** Proofs would be propagated through the Consensus Layer's peer-to-peer network, analogously to how blocks and attestations are handled.
- **Protocol inclusion (EIP-8025):** The EIP-8025 proposal[10] introduces two new modes for validators: *zkEVM Proof generating* and *Stateless validation*. Generator nodes would propagate proofs on dedicated CL subnets, while stateless nodes would consume them without the need for any external endpoint. Since proofs are optional, the EIP allows a gradual transition without impacting consensus, serving as a testbed for a future protocol-level requirement.

3. Data Availability via Portal Network. The Data Availability gap of the attester-only node could be bridged by integration with the Portal Network[11], a DHT-type decentralized network designed specifically to serve historical and state data to light clients. The Portal Network distributes the global state in fragments among participating nodes, enabling RPC queries such as `eth_getBalance` without any single node having to maintain the entire database. The integration of a ZeroNethermind client with a Portal node would allow combining stateless validation with the ability to respond to state queries, overcoming the attester-only limitation without reintroducing the storage requirements of a full node.

4. Deployment on mobile devices. The combination of $O(1)$ verification ($\sim$120 ms), RAM consumption below 500 MB, and the absence of disk I/O makes the deployment of a stateless validator on mobile devices conceivable. The Rust FFI bridge is already compilable for ARM64 architectures today, and the resource footprint measured in

Demo 2 is compatible with mid-range smartphones. A mobile validator would contribute to network decentralization in an unprecedented way, lowering the entry barrier to active consensus participation.

6.4 Final Considerations

The shift from the $O(N_{gas})$ paradigm to the $O(1)$ paradigm modifies the nature of the trade-off in the Blockchain Trilemma: rather than sacrificing scalability to preserve decentralization, the computational burden is delegated to specialized infrastructures (provers), distributing verification across a broader and more accessible validator base. The hardware requirement for a validator would go from 2 TB of NVMe SSD and 32 GB of RAM to a device with a few hundred megabytes of memory and a broadband connection. With the parallel adoption of PeerDAS for data availability and zkEVMs for validation, Ethereum could simultaneously achieve high throughput, decentralized consensus, and universally accessible validation.

This thesis has demonstrated that the Fort Mode component of this vision is technically achievable today, within a production client, with performance compatible with the protocol's time constraints. The remaining work lies in the maturation of the proving infrastructure, the reduction of proof generation latency, and their native integration into the consensus layer.

The architectural principle underlying this work is the transformation of validation from a computational power problem to a mathematical problem: Zero-Knowledge cryptography introduces a computational asymmetry such that verifying a block with ten thousand transactions requires the same resources as verifying a block with a single transaction. When this property is integrated at the protocol level, hardware requirements will cease to constitute an entry barrier and consensus participation will extend to a class of devices currently excluded from active validation.

References

[1] Jordi Baylina et al. ZisK: A fast zkVM architecture. [Repository GitHub], 2025.

[2] Vitalik Buterin. Possible futures of the Ethereum protocol, part 4: The Verge. [Articolo online], 2024.

[3] Stefanos Chaliasos, Denis Firsov, and Benjamin Livshits. Towards a formal foundation for blockchain zk rollups. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 2714–2728, 2025.

[4] Panagiotis Chatzigiannis, Foteini Baldimtsi, and Konstantinos Chalkias. Sok: Blockchain light clients. In *International Conference on Financial Cryptography and Data Security*, pages 615–641. Springer, 2022.

[5] Justin Drake. Keynote + demo: zkEVM attesting. [Video], 2025. Ethereum Foundation, Devconnect.

[6] Justin Drake. Lean Ethereum. [Articolo online], 2025. Ethereum Foundation Blog.

[7] ETH Applied Cryptography Team. Ethereum zkEVM book. [Sito web], 2026.

[8] Ethereum Community. EIP-4762: Statelessness gas cost changes. [Specifica online], 2022.

[9] Ethereum Community. EIP-7732: Enshrined proposer-builder separation (ePBS). [Specifica online], 2024.

[10] Ethereum Community. EIP-8025: Optional execution proofs. [Specifica online], 2026.

[11] Ethereum Community. Portal network. [Documentazione online], 2026.

[12] Ethereum Foundation. Ethproofs — learn. [Sito web], 2023.

[13] Ethereum Foundation. L1 strawmap — Ethereum draft roadmap. [Sito web], 2026.

[14] Ethereum Foundation zkEVM Team. L1-zkevm roadmap 2026: Integrating zkevm proofs into ethereum's core protocol. [Specifica online], 2026.

[15] Google. cAdvisor: Container resource usage and performance analysis. [Repository GitHub], 2021.

[16] Julian. An Ethereum prover market proposal. [Specifica online], 2025.

[17] Kurtosis Technologies. Kurtosis — ethereum network testing. [Sito web], 2024.

[18] L2BEAT Team. L2BEAT stages: A framework to evaluate rollup decentralization. [Sito web], 2024.

[19] Nethermind. Database — Nethermind documentation. [Documentazione online], 2026.

[20] Nethermind. System requirements — Nethermind documentation. [Documentazione online], 2026.

[21] Orochi Network. Blockchain trilemma: Balancing decentralization, security, and scalability. [Articolo online], 2025.

[22] Prometheus Authors. Prometheus — monitoring system and time series database. [Sito web], 2025.

[23] Giovanni Quattrocchi, Filippo Scaramuzza, and Damian A Tamburri. The blockchain trilemma: an evaluation framework. *IEEE Software*, 41(6):101–110, 2024.

[24] Stateless Consensus. Ethereum stateless book. [Sito web], 2025.

[25] Succinct Labs. SP1: A performant, 100% open-source zkVM. [Repository GitHub], 2024.

[26] Justin Thaler. Proofs, arguments, and zero-knowledge. *Foundations and Trends in Privacy and Security*, 4(2–4):117–660, 2022.

[27] Gavin Wood et al. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 151(2014):1–32, 2014.